End-of-Studies Project (PFE)

A Privacy-First, No-Code AI Platform
for Tabular Machine Learning, Generative AI, and Deep Learning

[Student Name]
Supervised by [Supervisor Name]

May 20, 2026

Abstract

This report describes a no-code AI platform I built over eight sprints as an end-of-studies project. The platform lets non-technical users upload their data, sketch a pipeline on a visual canvas, and train a model — with one firm constraint: nothing ever leaves the machine. Three workflows sit behind the same canvas. The first is classical tabular ML, with fourteen algorithms spanning classification, regression, clustering, and forecasting. The second is local Generative AI through a Retrieval-Augmented Generation chat that answers from the user's own documents, powered by Ollama and `pgvector`. The third, added during the final sprint, is deep-learning image classification using a small catalogue of CNN architectures sized to fit on the GTX 1660 Super (6 GB VRAM) that served as the demo box. The chapters that follow walk through each sprint's design choices, the things that broke on the first try, and the handful of architectural conventions I ended up leaning on throughout: Celery for anything CPU-bound, Mongo for anything that genuinely resists a schema, and a strict gateway-only header-injection scheme that gave the microservices a zero-trust boundary without any of the usual gymnastics.

Contents

General Introduction

Machine learning has become routine in almost every industry, yet the tooling around it still tends to assume one of two extremes. On one end sit the cloud-managed services — DataRobot, Vertex AI, Azure ML Studio — which hide the complexity behind polished interfaces but require users to ship their data into someone else's infrastructure. On the other end live the open notebooks where everything is possible and nothing is easy: Jupyter, Colab, raw Python. The space between those two poles, where a non-technical user might want to build a model without either learning Python or handing over their data, is surprisingly thin.

That gap is what this project sets out to address. The goal was a platform that gives a domain expert — a clinician, an analyst, a small business owner — enough visual scaffolding to build, train, and inspect a model on their own machine. Not a tutorial, not a hosted SaaS, but a real piece of software that runs locally, accepts local data, and never phones home to a vendor API.

Building this turned out to be more interesting than I expected. Three distinct workloads — tabular ML, RAG-style generative AI, and image classification — impose three different operational profiles, and fitting them into a single coherent product meant making choices that weren't obvious upfront. Some of those choices held up well. A few didn't, and the chapter on reflections is honest about which is which.

The report is organised as follows. The opening chapters set the scene: the project's context and scope, a quick look at the broader no-code ML landscape, and the methodology I used to keep eight sprints moving on a solo timeline. The middle chapters walk through each sprint in turn, from the authentication-and-gateway foundation up through the deep learning module added in the final stretch. The closing chapters cover testing and deployment, then step back to discuss what worked, what didn't, and what I'd build differently with another quarter to spend.

Chapter 1

Project Context and Scope

Introduction

This first chapter lays out the question the project tries to answer, the audience it imagines, and the scope I committed to before any code was written. It also gives a high-level tour of the platform so the later chapters have something concrete to refer back to.

1.1 Why no-code, and why local?

The original question was narrow on purpose: *can a non-technical analyst go from a CSV file to a deployed model without ever shipping their data to someone else's cloud?* The first half of that question has been answered many times over. DataRobot, Azure ML Studio, and Vertex AI all give you a drag-and-drop pipeline builder of one kind or another. But every one of them assumes the user is fine uploading their dataset to a vendor bucket. For a hospital, a bank, or any organisation that sits on PII or trade secrets, that assumption isn't acceptable — and it's worth noting that it's not a contractual problem, it's a regulatory one. The data simply cannot leave.

The second half of the question — can it actually run locally? — has only recently become realistic. Quantised LLMs that fit inside 4–6 GB of VRAM (Llama 3.2 3B, Phi-3 Mini, Gemma 2) opened the door to local generative AI. PyTorch has supported local image classification for years, but only in the last couple of generations of consumer GPUs has a 6 GB card been enough to fine-tune small CNNs without contortion. The platform I built sits inside both of those windows: it runs Ollama for chat, sentence-transformers for embeddings, scikit-learn / H2O / XGBoost for tabular ML, and PyTorch for the image-classification workflow that landed in the final sprint.

1.2 What the platform does

A user, once signed in, can:

- Upload tabular data (CSV, Excel), pull it from a Postgres or MySQL connector, or upload a zip of images organised by class folder.
- Inspect a profile of the dataset — missing values, skew, outliers, target imbalance, correlations — before doing anything else with it.

- Build a pipeline visually on a React Flow canvas, in one of three modes:
  - **Traditional ML** — fourteen supervised and unsupervised algorithms spanning classification, regression, clustering, and time-series forecasting.
  - **Generative AI** — index documents into a local vector store and chat with a RAG pipeline.
  - **Deep Learning** — image classification on a fixed catalogue of three CNN architectures (LeNet, a custom ResNet-9, and MobileNet-V3-Small with optional ImageNet transfer learning).
- Train, evaluate, download the trained artefact, and — for image models — run live inference on a hand-uploaded test image.
- Collaborate with teammates inside a company workspace, with role-based access control, real-time canvas presence, threaded chat per pipeline, and one-click Google Meet links.

Alongside the user-facing application sits a super-admin role with its own dashboard for user management, audit logs, queue health, hardware monitoring, and a "view-as-user" impersonation flow gated by a hard five-minute TTL.

1.3 Functional and non-functional requirements

1.3.1 Functional

- Tabular ML pipelines: dataset → train → evaluate, backed by fourteen algorithms.
- RAG pipelines: document → vector store → chat, end to end on the local box.
- DL pipelines: image dataset → CNN architecture → training loop, again fully local.
- User authentication with email/password and optional TOTP-based two-factor authentication.
- Stripe-backed billing across free, solo, and company tiers, with super-admin overrides on per-user quotas.
- Real-time presence and shared editing on the pipeline canvas.

1.3.2 Non-functional

- **Data sovereignty.** No outbound calls to AI vendors. The only third-party service in the deployment graph is Stripe, and Stripe is optional — without an API key the platform still works on the free tier.
- **Modularity.** The system splits into five independent Python services so that a long XGBoost run can never block an authentication request.
- **Reproducibility.** Every dataset version is profiled, every training run is recorded, every model artefact is versioned and downloadable.

- **Hardware constraints.** The whole stack runs on a single consumer machine. My development box — a laptop with 16 GB of RAM and a GTX 1660 Super GPU — was picked deliberately, because it sits close to the floor of what someone might realistically deploy on.

1.4 High-level architecture

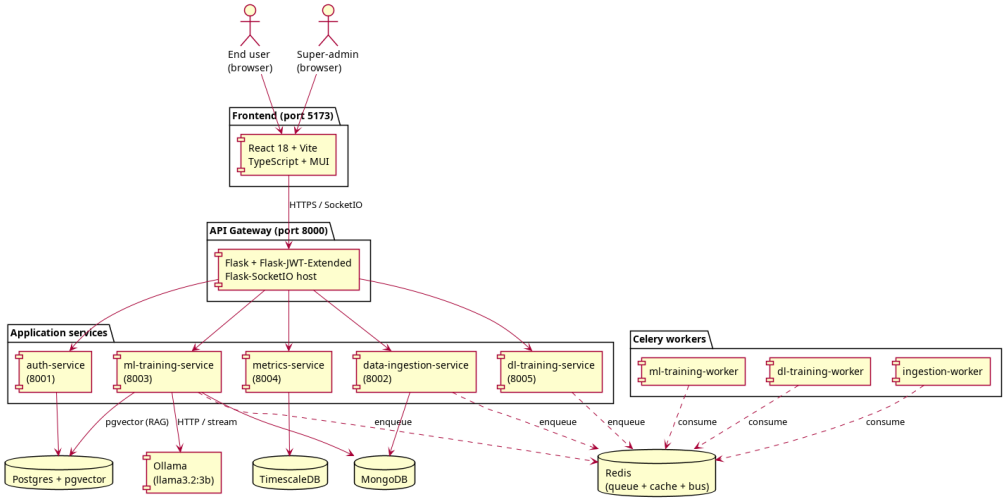


Figure 1.1: Overall system architecture

The overall shape is conventional: a React single-page app talks to a Flask API gateway, which decodes JWTs, injects authoritative `X-User-*` headers, and proxies the request to one of five upstream services. What is less conventional is the strict locality. Every model server, every vector store, every queue runs inside the same Docker Compose stack. There is no cloud the platform can fall back to if something local fails, and that was a deliberate design constraint rather than an oversight.

Conclusion

The chapter framed the problem the rest of the report addresses — how to make ML approachable without sacrificing data sovereignty — and sketched the platform that emerged in response. The following chapter turns to the broader landscape to explain why this question is harder to answer with off-the-shelf tools than one might initially assume.

Chapter 2

Background and Related Work

Introduction

Before getting into how the platform was built, it's worth a brief look at why nothing already on the market quite fits the problem. The no-code ML space is crowded, but the products in it cluster into families that all share the same blind spot.

2.1 The no-code ML landscape

Three categories of product dominate the space:

- **Cloud-native AutoML platforms** (DataRobot, H2O Driverless AI, Azure ML Studio, Google Vertex AI). Highly polished, exhaustive algorithm catalogues, and almost invariably SaaS-only — meaning their privacy story ends at "trust our infrastructure". For some buyers that's fine; for regulated industries it isn't.
- **Open-source notebooks** (Jupyter, Google Colab, DataLab). Powerful for experts, but every workflow eventually bottoms out in raw Python. That's exactly the friction the no-code category was invented to remove.
- **Visual ETL tools** (KNIME, Alteryx, RapidMiner). Closer in spirit to what I built, but their centre of gravity is data preparation rather than model training, and their LLM integrations tend to route requests back through OpenAI.

The platform I built tries to sit at the intersection of those three: a visual canvas in the spirit of KNIME, an algorithm catalogue at the scale of an AutoML service, and a deployment story modelled on a self-hosted notebook. Nothing leaves the box.

2.2 Why local LLMs are now realistic

When the project started I quietly assumed local generative AI would be too slow to ship as a real feature. Two technical shifts changed that:

1. **Aggressive quantisation.** Four-bit quantisation (`Q4_K_M`) shrinks a three-billion-parameter model from roughly 12 GB of weights down to about 2 GB, while preserving most of the original quality on conversational tasks. The trade-off is real but, for chat, it isn't fatal.

2. **First-class GPU inference runtimes for consumer hardware** (Ollama, llama.cpp, vLLM). A 1660 Super, sporting 6 GB of VRAM and 30 TFLOPS of FP16 throughput, can serve roughly 30 tokens per second on a 3B model — which is enough latency budget for an interactive RAG chat.

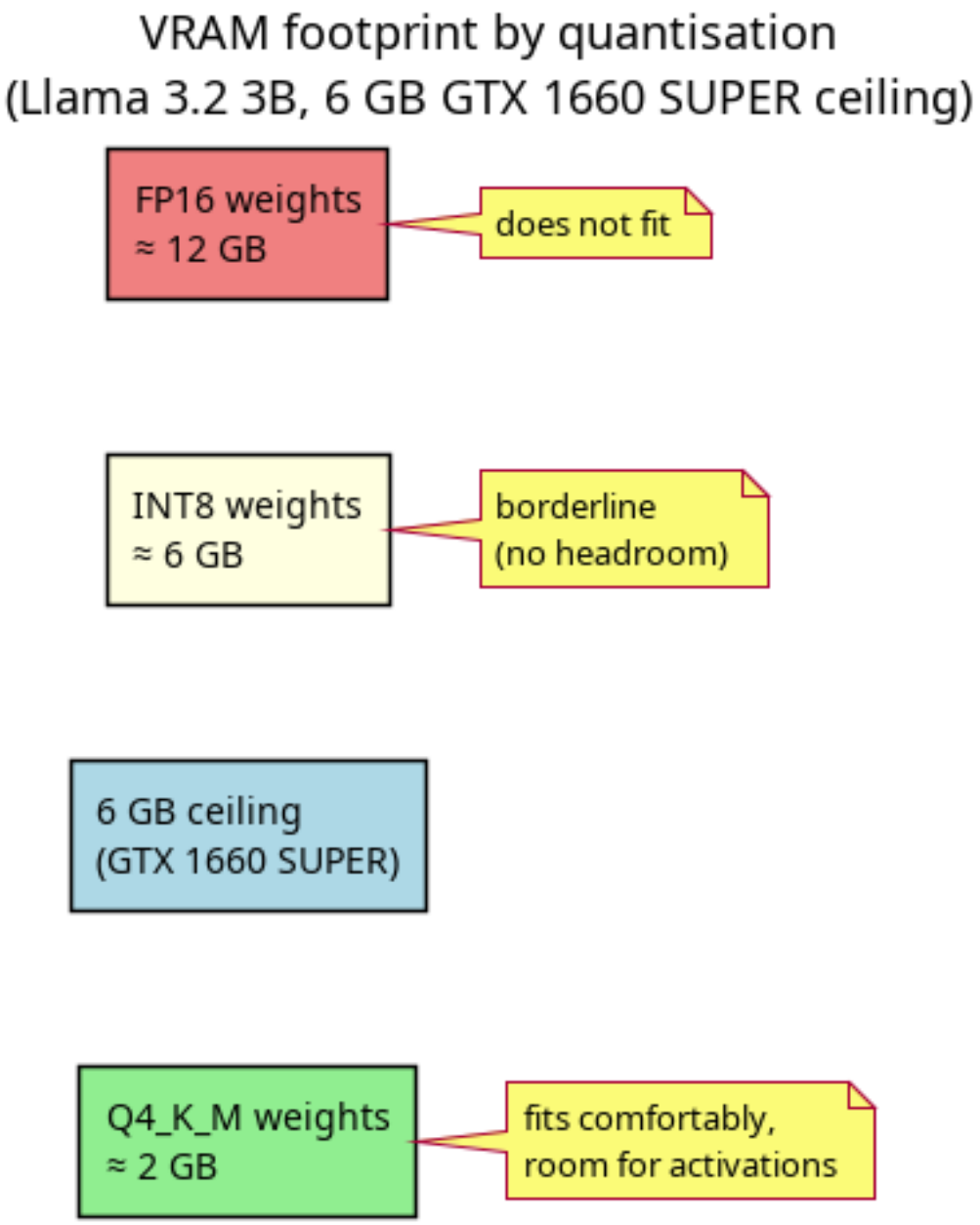


Figure 2.1: Quantisation → VRAM mapping

Conclusion

Cloud platforms solve the wrong half of the problem, notebooks demand too much of the user, and visual ETL tools aren't really aimed at model training. The shift in quantisation and consumer GPU inference is what makes this project viable today; it would not have

been a year or two ago. The next chapter turns to the methodology I used to translate that opening into a working product.

Chapter 3

Methodology and Working Environment

Introduction

Working alone for several months on a project this broad demanded some discipline. This chapter explains the methodology I followed, the technology choices that anchored the build, and the layout of the services those choices produced.

3.1 Why Agile, sprint-shaped

The temptation, when you're the only person on a project, is to skip ceremony — no standups to attend, no retros to chair, no Jira to update. I resisted that for two reasons. First, ML capacity planning is genuinely unpredictable. A feature that looked easy on paper ("add Prophet for time-series forecasting") ended up requiring `cmdstan` to be compiled from source, which doubled the size of one of my Docker images. That kind of surprise compounds if you don't have a sprint boundary to absorb the slip. Second, I needed checkpoints I could demo to my supervisor without hand-waving. Without sprint structure I would have ended up with a single "everything works at once" deliverable, no story to tell along the way, and no fallback position if a late feature fell through.

So the eight sprints below each ended with a runnable demo and a self-contained git tag.

3.2 Technology choices

- **Frontend: React 18 + Vite + TypeScript + MUI.** React for the ecosystem, Vite because cold starts under `create-react-app` were embarrassingly slow, MUI because it ships accessible components out of the box — a point that matters for the company-tier sale.
- **Backend: Flask + Celery + Flask-SocketIO.** Flask was the lowest-common-denominator choice for a microservice. FastAPI would have given me typed routes for free, but every template I started with assumed Flask, and going back through five rewrites just for typing wasn't a fight I wanted to have. Celery handles every

CPU-bound task: profiling, preprocessing, H2O training, RAG ingestion, image extraction, and PyTorch training each live on their own queue.

- **Postgres + pgvector for relational and vector data.** Co-locating the vector store with the auth schema let me avoid spinning up a sixth backing service. `pgvector`'s IVFFlat index is more than enough for the few hundred thousand chunks that a single-tenant local deployment is ever likely to hold.
- **MongoDB for everything schemaless.** Datasets, pipelines, model versions, task results — anything whose shape evolves between sprints lives in Mongo. It's the path of least resistance for objects that genuinely resist a fixed schema.
- **Redis for queue + cache + socket message bus.** Three logical databases on the same instance: DB 0 for Celery, DB 1 for Celery results, DB 2 for the SocketIO message queue.
- **Ollama for local LLM inference.** The alternative was calling `llama.cpp` directly. That would have given me finer control at the cost of building my own HTTP shim around it; Ollama's REST API is good enough that the trade wasn't close.
- **PyTorch for image classification.** Added in Sprint 8. The CUDA wheels are downloaded from PyTorch's CUDA 12.1 index inside the Dockerfile so the host only needs the NVIDIA driver plus `nvidia-container-toolkit`.

3.3 Microservice topology

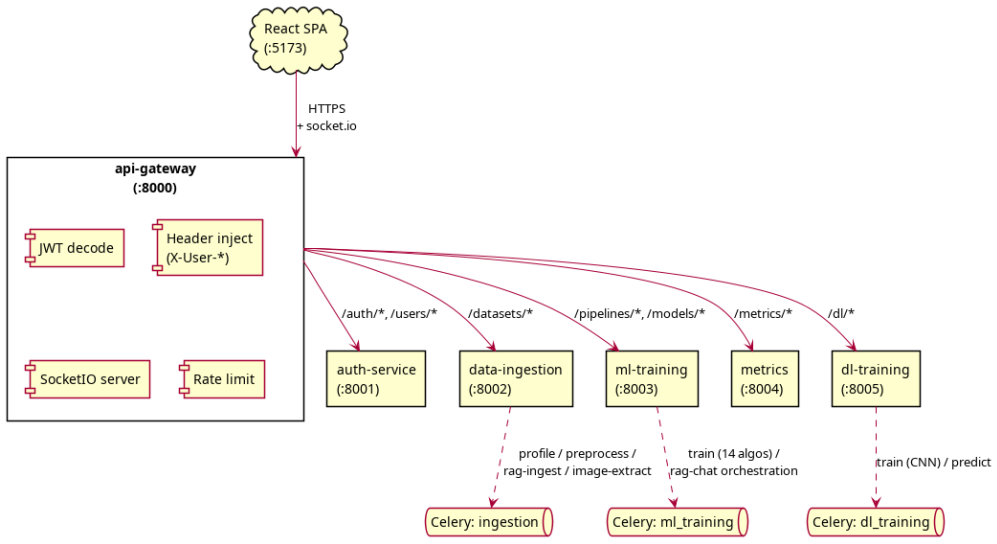


Figure 3.1: Microservice topology

- `api-gateway` (port 8000) — decodes JWTs, injects `X-User-Id` / `X-User-Tier` / `X-User-Role` headers, proxies to the upstream services. It also hosts the SocketIO server for chat and training presence.

- `auth-service` (port 8001) — user accounts, billing, admin operations, project and company ACL.
- `data-ingestion-service` (port 8002) — file uploads, SQL and S3 connectors, profiling, preprocessing, RAG ingestion, image-dataset extraction.
- `ml-training-service` (port 8003) — the fourteen tabular algorithms, model registry, RAG chat orchestration.
- `metrics-service` (port 8004) — TimescaleDB-backed hardware-telemetry collection (added in Sprint 7).
- `dl-training-service` (port 8005) — PyTorch image classification (added in Sprint 8).

3.4 CI/CD

A GitHub Actions workflow runs four jobs in parallel on every push:

- Backend lint — `black`, `isort`, `flake8` across all five Python services.
- Frontend lint — `eslint` and `tsc` in strict mode.
- Backend tests — a PostgreSQL + Mongo + Redis stack is spun up as services, then `pytest` runs against each microservice. For `dl-training-service`, an extra step installs torch's CPU-only wheels so the suite doesn't pull a gigabyte of CUDA binaries it has no way to use on a CI runner.
- Frontend tests — `vitest` with coverage upload.

End-to-end, CI runs in roughly six minutes for a frontend-only change, nine minutes for a backend change, and fourteen minutes when both sides move at once. Slow enough that I noticed; not so slow that I bypassed it.

Conclusion

The methodology and toolchain described here held throughout the eight sprints. Some of these choices — Flask in particular — are ones I'd revisit if starting over, but they did get me to a working system without rework. The chapters that follow take each sprint in turn and explain what was built, why, and what broke along the way.

Chapter 4

Sprint 1: Authentication and the Gateway

Introduction

Sprint 1 was the foundation sprint. Nothing here is glamorous — no charts, no dashboards, no models — but the design choices made in these two weeks shaped every later sprint, and a couple of them ended up being the quiet load-bearing decisions of the entire project.

4.1 Goal

The goal was a stateless JWT authentication scheme, a Redis-backed token revocation list, role-based access control with super-admin and tenant roles, and a gateway that strips client-supplied identity headers before injecting authoritative ones from the decoded JWT. None of this is novel on its own. Wiring it together correctly is what matters.

4.2 Design

4.2.1 Why an opaque-but-decoded gateway?

Two reasonable architectures sat on the table:

1. Have the gateway hit `auth-service` on every request to validate the JWT.
2. Have the gateway hold the same `JWT_SECRET_KEY` as `auth-service` and decode the token in-process.

I went with the second, and I'd make the same call again. The gateway becomes a hot path; a synchronous HTTP round-trip per request adds latency that compounds badly under load. The price is that the gateway needs the same signing secret as the auth service — which, in a shared environment file deployment, is a non-issue. In a horizontally scaled production setup it would be more involved; for the demo target it's the right trade.

4.2.2 Header-injection rules

The gateway strips any client-supplied `X-User-*`, `X-Company-*`, or `X-Project-*` header before forwarding, then re-adds them from the decoded JWT. This is, without exaggeration, the most important single security invariant in the whole system. **Downstream services trust `X-User-Id` unconditionally, which means an attacker who can supply that header without going through the gateway can impersonate anyone.** The strip-then-inject pattern is deliberate, well tested, and not negotiable.

4.3 Implementation

```python
def _forward(upstream_base, path, require_auth=True, timeout=30):
    if require_auth:
        try:
            verify_jwt_in_request()
            claims = get_jwt()
            g.user_id = get_jwt_identity()
            g.user_role = claims.get("role", "")
            g.user_tier = claims.get("tier", "free")
        except Exception as e:
            return jsonify({"error": "unauthorized", "message": str(e)
    }), 401

    # Strip every client-supplied identity header before forwarding.
    _STRIP_PREFIXES = ("x-user-", "x-company-", "x-project-")
    headers = {
        k: v for k, v in request.headers
        if k.lower() not in _HOP_BY_HOP
        and k.lower() != "host"
        and not k.lower().startswith(_STRIP_PREFIXES)
    }
    # Re-inject the authoritative ones from the decoded JWT.
    if require_auth or hasattr(g, "user_id"):
        headers.update(get_forwarded_headers())

    resp = requests.request(
        method=request.method, url=f"{upstream_base.rstrip('/')}{path}"
    ,
        headers=headers, data=request.get_data(), stream=True, timeout=
    timeout,
    )
    return _stream_response(resp)
```

Listing 4.1: Gateway proxy with header injection (excerpt from `api-gateway/app/routes/proxy.py`)

4.4 Security posture

- Access tokens have a fifteen-minute TTL. Refresh tokens live as HttpOnly cookies and rotate on every refresh; the previous token's JTI lands in the Redis blacklist so a stolen refresh token can actually be revoked.

- Passwords use `bcrypt` at cost 12. Refresh tokens themselves are stored as SHA-256 hashes in Postgres — the database-breach scenario yields a hash, not a usable token.
- TOTP-based two-factor authentication is optional per user; super-admins can require it per-tier from the admin panel.
- Per-IP and per-user rate limits sit at the gateway. The per-user limit defaults to 600 req/min in development, deliberately loose because React 18 Strict Mode double-invokes `useEffect` and a tighter cap was producing spurious 429s.

4.5 Outcome

By the end of the sprint the platform had end-to-end authentication, a working refresh-token loop, and the gateway pattern that the next seven sprints would build on top of. The authoritative-headers convention turned out to be one of the project's most quietly useful design decisions: every subsequent service trusts `X-User-Id` without a second thought, because the gateway is the only path along which it can be set.

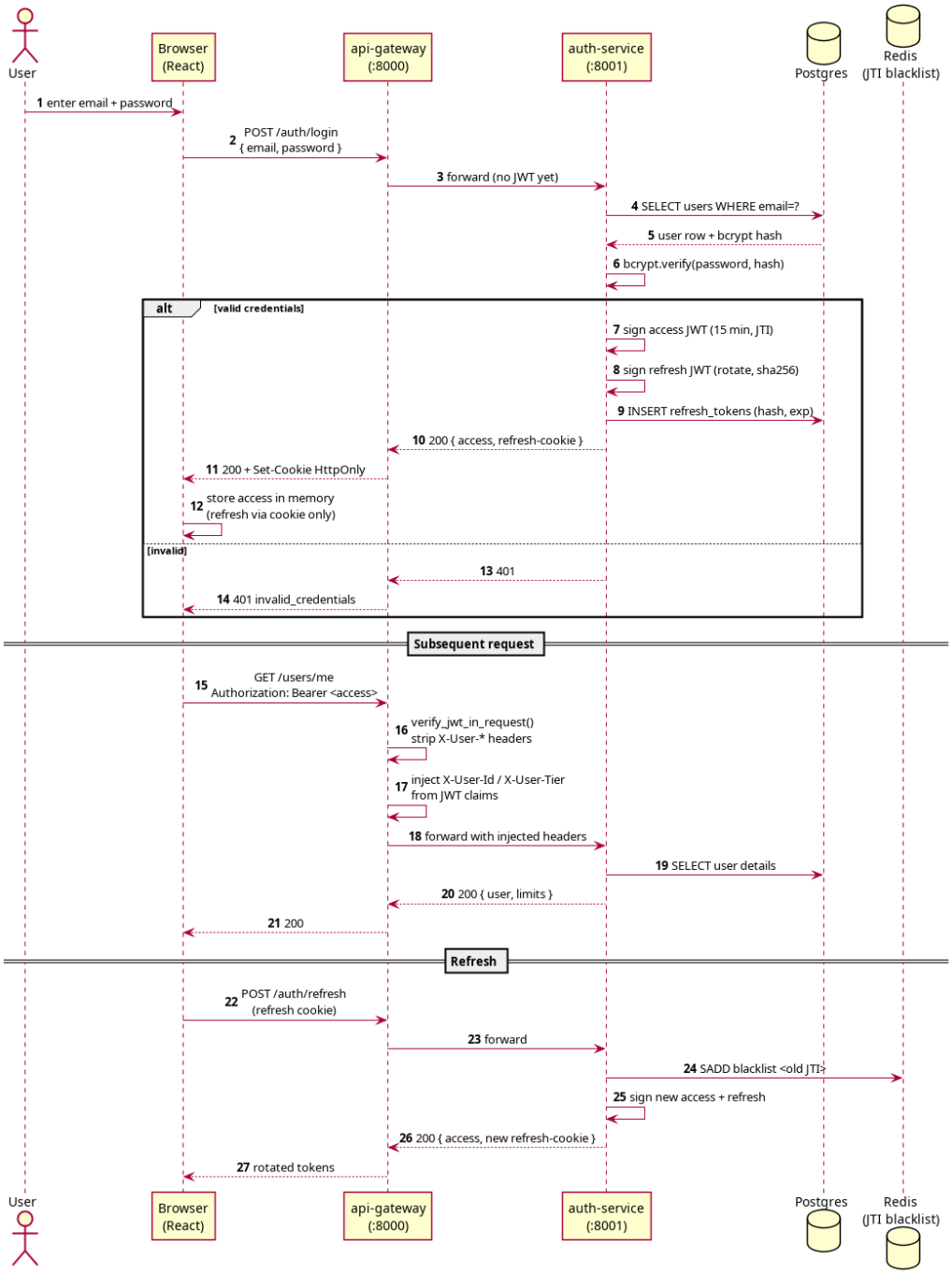


Figure 4.1: Login + JWT issuance sequence diagram

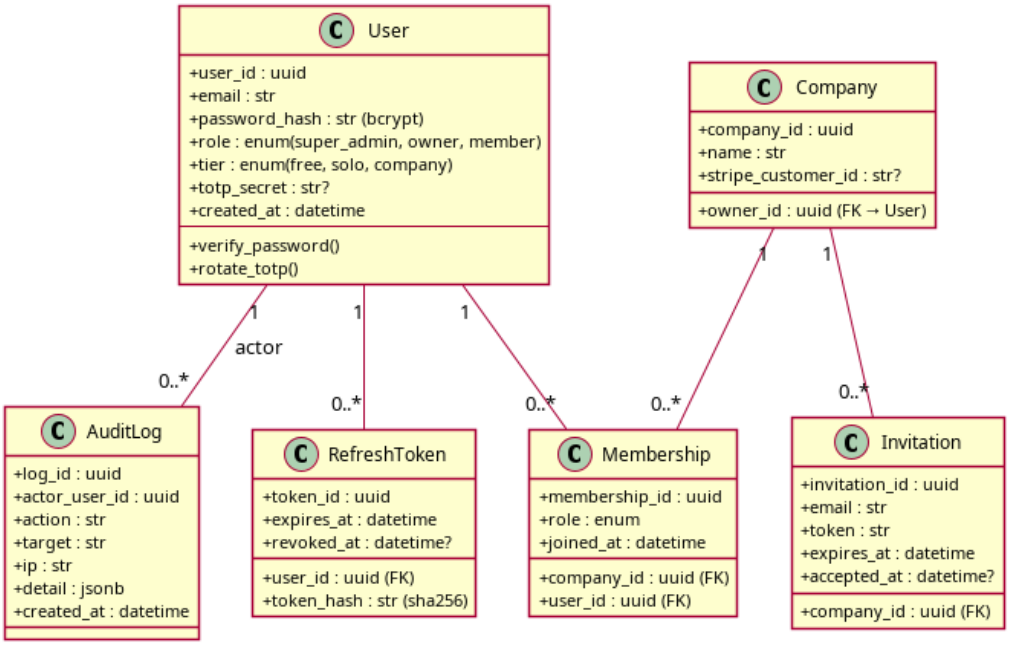


Figure 4.2: Class diagram for Sprint 1

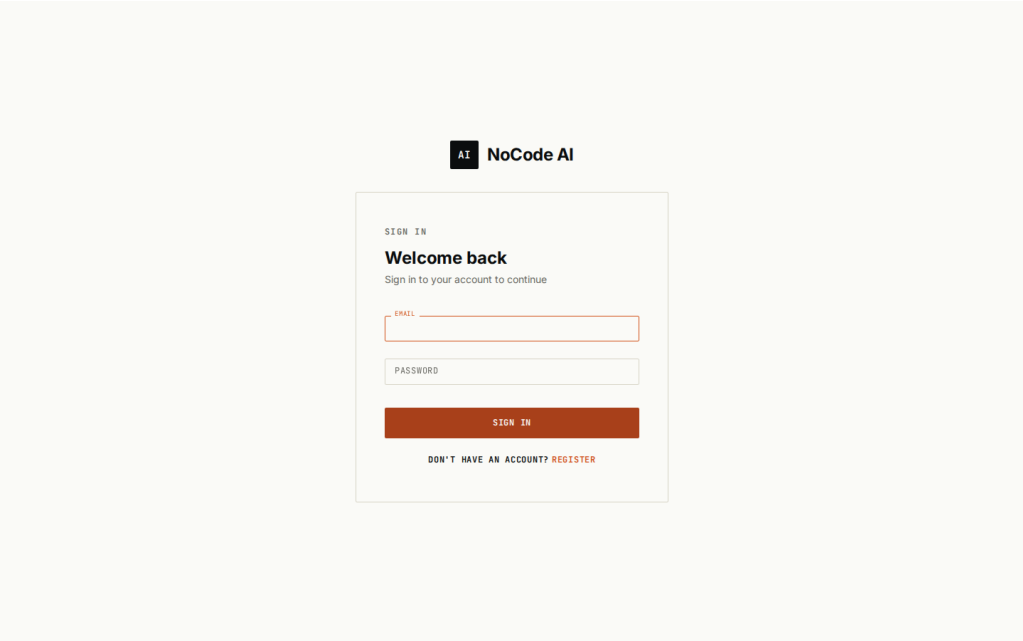


Figure 4.3: Login screen

Chapter 5

Sprint 2: Data Ingestion and Profiling

Introduction

With authentication in place, Sprint 2 turned to the next obvious need: getting data into the platform. The deliverable was the `data-ingestion-service` — file uploads, SQL connectors, asynchronous profiling, and a preprocessing pipeline that splits data into train, validation, and test sets and saves them as Parquet for the downstream training services to consume.

5.1 Goal

End-to-end: a user uploads a CSV through the browser, the file lands on disk, a Celery task profiles it asynchronously, and the frontend sees the profile light up as it completes. The path needed to be both fast and unbreakable in the face of large or messy files.

5.2 Design

5.2.1 The shared-volume convention

My first instinct was to have the ingestion service stream files to the worker over the network. I started building that, then realised halfway in that my Docker Compose stack already had a named volume (`uploaded_files`) that both the API container and the worker container could mount. Mounting it in both places collapsed the streaming-upload protocol into a one-line write: the API saves to `/uploads/<id>.csv`, the worker reads from the same path. That pattern then became the convention for every binary artefact in the system — trained models, RAG documents, image-dataset extracts. Sometimes the right design is the one you almost overlooked because it was already sitting in front of you.

5.2.2 Why MongoDB for dataset metadata

A dataset's profile is fundamentally schemaless. A CSV with two numeric columns produces a profile that looks nothing like the profile of a CSV with thirty mixed-type columns including a target. I spent the better part of a day considering a Postgres JSONB column for this and ultimately preferred Mongo's storage model. It's lossless, it's queryable

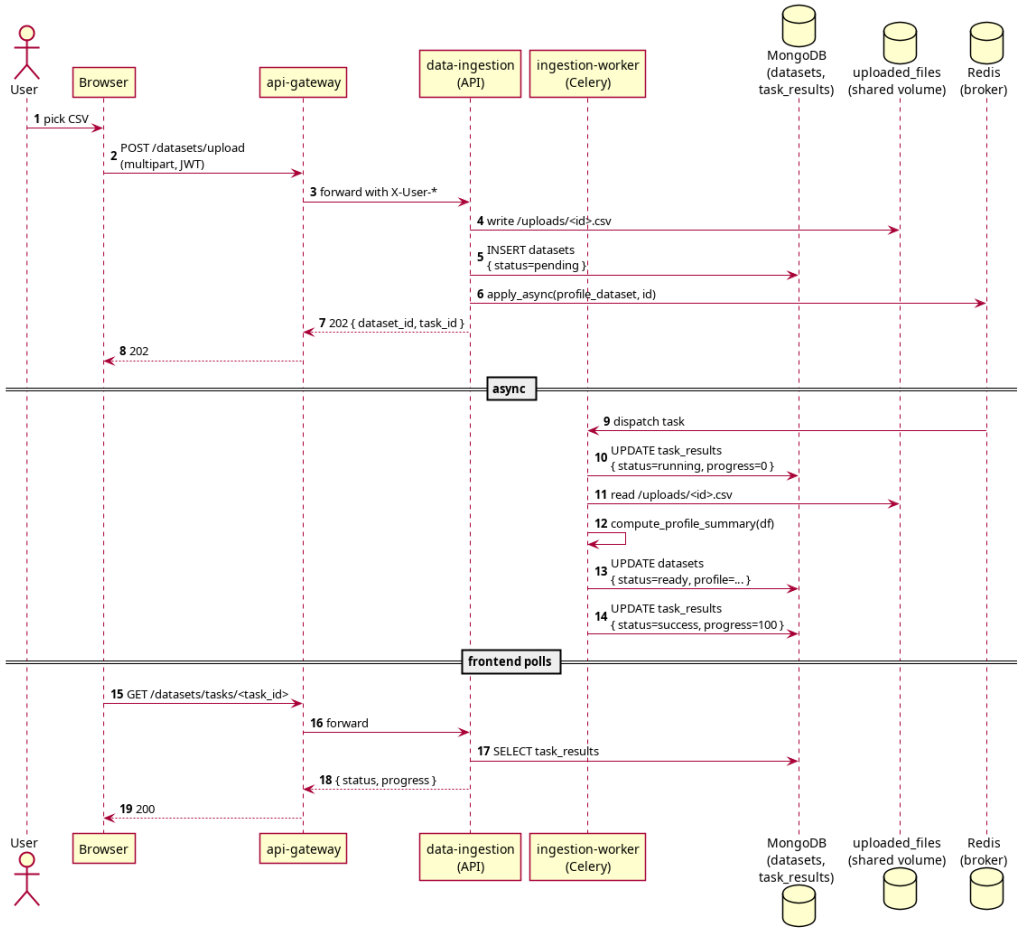


Figure 5.1: Asynchronous upload sequence

enough for the use case, and it doesn't impose a schema migration every time I want to add a new statistic.

5.3 Implementation

```python
@celery.task(name="app.tasks.profiling.profile_dataset", bind=True)
def profile_dataset(self, dataset_id: str):
    client = MongoClient(os.environ["MONGO_URL"])
    db = client[os.environ.get("MONGO_DB", "nocode_ingestion")]
    datasets = db["datasets"]
    task_results = db["task_results"]

    task_results.update_one(
        {"task_id": self.request.id},
        {"$set": {"status": "running", "progress_pct": 0}},
        upsert=True,
    )
    doc = datasets.find_one({"dataset_id": dataset_id})
    df = load_dataframe(doc["file_path"])

    summary = compute_profile_summary(df, target_column=doc.get("
   target_column"))

```

```
    datasets.update_one(
        {"dataset_id": dataset_id},
        {"$set": {
            "status": "ready",
            "profiling_summary": summary,
            "row_count": len(df),
            "column_count": len(df.columns),
        }},
    )
    task_results.update_one(
        {"task_id": self.request.id},
        {"$set": {"status": "success", "progress_pct": 100}},
    )
```

Listing 5.1: Profiling task (excerpt from `data-ingestion-service/app/tasks/profiling.py`)

5.3.1 SQL connectors and Fernet encryption

Users can configure a Postgres or MySQL connector inside the platform; the credentials are encrypted with a Fernet key (AES-128-CBC plus HMAC) held in the service's environment, then stored in Mongo. The same key is reused for any other secret the ingestion service needs to hold, and rotating it is a documented operations procedure rather than a fire-fight. SQL queries themselves run through SQLAlchemy with parameter binding, which makes SQL injection structurally impossible rather than merely unlikely.

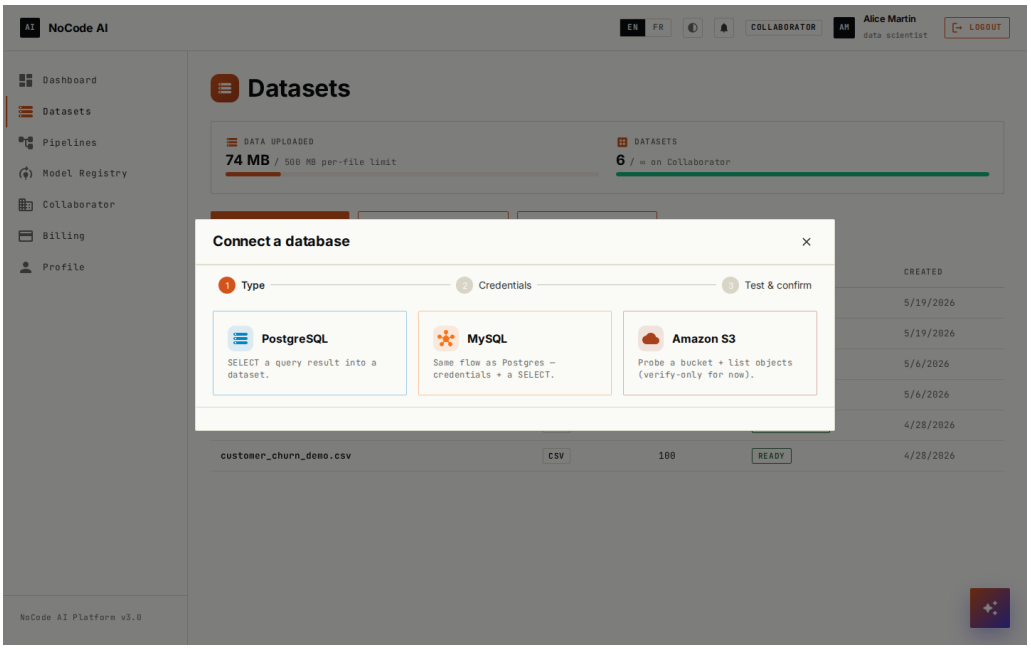


Figure 5.2: Connector wizard screenshot

5.4 Outcome

By sprint's end the platform could ingest CSVs, profile them asynchronously, preprocess them into clean Parquet splits, and surface all of that to the frontend. Everything past

this point assumed the ingestion path was a solved problem, which by and large it was. The only ingestion-related work that resurfaced later was the SQL probe step (Sprint 4) and the image-dataset path that arrived with deep learning in Sprint 8.

Chapter 6

Sprint 3: Visual Pipeline Editor and Tabular Training

Introduction

Sprint 3 was the largest single sprint of the project. It pulled together three big pieces — the visual pipeline editor, the `ml-training-service` with its fourteen tabular algorithms, and the live metric visualisations — into a coherent flow the user could actually follow from canvas to model.

6.1 Goal

The deliverables: a directed acyclic graph editor on a React Flow canvas, the `ml-training-service` hosting all fourteen algorithms, a model registry with versioning, and live metric streaming from the training process back to the canvas.

6.2 Design

6.2.1 Graph schema

A pipeline is stored in Mongo as a document with two arrays:

```
{
  "pipeline_id": "...",
  "user_id": "...",
  "type": "ml" | "rag" | "dl",
  "nodes": [
    { "node_id": "n1", "type": "dataset", "position": {"x": 60, "y": 120},
      "data": { "dataset_id": "..." } },
    { "node_id": "n2", "type": "train", "position": {"x": 320, "y": 120},
      "data": { "algorithm": "xgboost", "task_type": "classification",
                "hyperparams": {...}, "target_column": "..." } },
    { "node_id": "n3", "type": "evaluate", "position": {"x": 580, "y": 120},
      "data": {} }
  ],
```

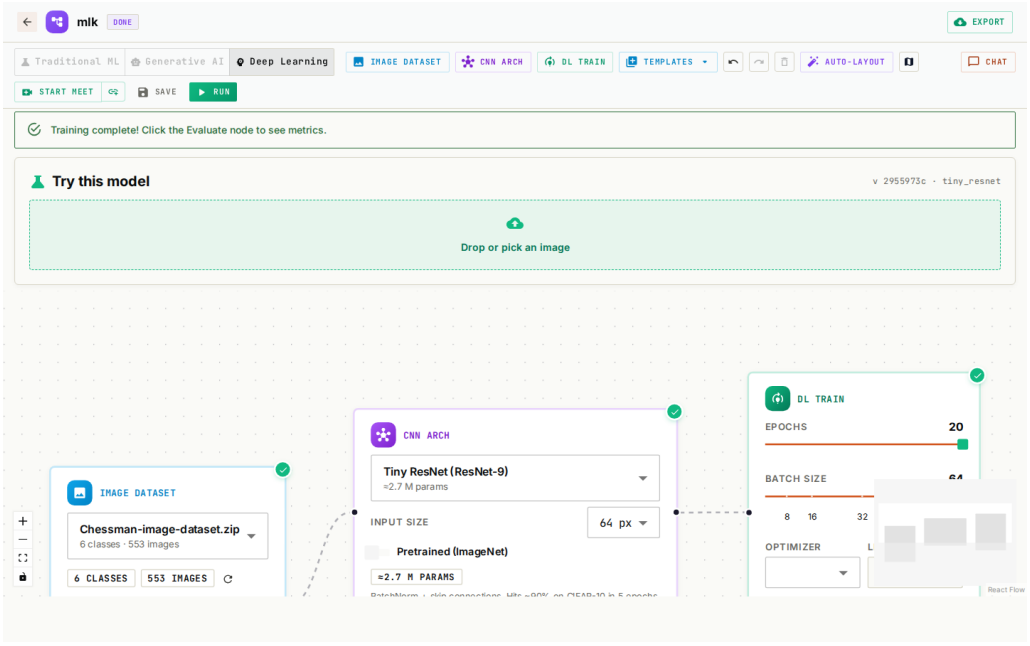


Figure 6.1: Pipeline editor screenshot — ML mode

```
  "edges": [
    { "source": "n1", "target": "n2" },
    { "source": "n2", "target": "n3" }
  ],
  "status": "draft"|"running"|"done"|"error"
}
```

Listing 6.1: Pipeline document shape

6.2.2 The `BaseMLModel` abstraction

Fourteen algorithms with one orchestrator implies an abstraction. Rather than over-engineer it, I went with a deliberately small abstract base class: each algorithm subclass exposes `train`, `predict`, and `evaluate` with a shared signature, whether it's wrapping scikit-learn, XGBoost, LightGBM, CatBoost, or Prophet. The training Celery task is algorithm-agnostic — it constructs a model from a registry, trains it, asks it for metrics, and serialises the estimator. Adding a new algorithm is, in practice, a one-file change.

```
class BaseMLModel(ABC):
    def __init__(self, hyperparams: dict[str, Any]):
        self.hyperparams = hyperparams
        self._estimator = None

    @abstractmethod
    def train(self, X_train, y_train=None) -> None: ...
    @abstractmethod
    def predict(self, X) -> Any: ...
    @abstractmethod
    def evaluate(self, X_test, y_test=None) -> dict[str, Any]: ...

    @property
    def estimator(self):
```

```
15          return self._estimator
```

Listing 6.2: Algorithm-agnostic training core

The registry mapping algorithm strings to subclasses lives in `ml-training-service/app/services/`

6.2.3 H2O for the heavy algorithms

For XGBoost, GBM, and Random Forest, I used H2O-3 instead of the scikit-learn implementations. H2O's algorithms run faster on tabular data, and they expose feature importance and SHAP values out of the box — which, on its own, removed an entire subtask from Sprint 4. Not every dependency pays for itself; this one did.

6.3 Frontend canvas

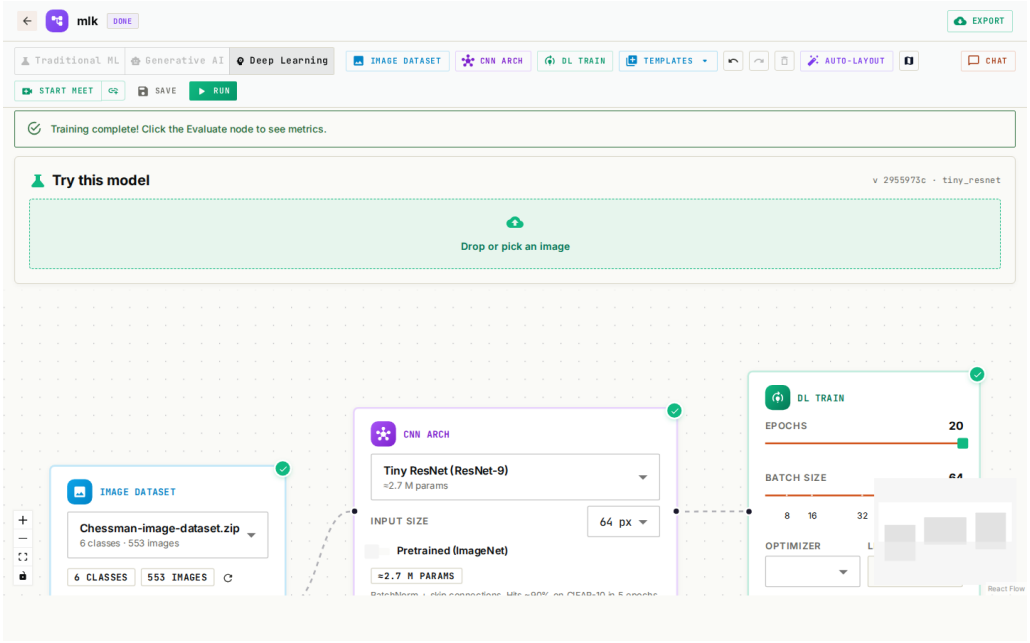


Figure 6.2: Node component anatomy

The canvas is a React Flow instance with custom node components. Each node type validates itself against the surrounding graph: the `Train` node shows an error badge if no `Dataset` node is upstream; the `Evaluate` node shows a warning if no training run has produced a version yet. The validation function is pure — `(node, edges, allNodes)` → `{status, message}` — which keeps it trivially testable.

6.4 Live training visualisation

Training emits per-step metric points over the SocketIO message queue. The `LiveTrainingChart` component subscribes to the pipeline's room (`pipeline_<id>`) on the `/training` namespace and plots the points on a Plotly scatter. The Y-axis is auto-scaled because progress percentage (0–100) and metric values (typically 0–1) live on different ranges — progress on the primary axis, metrics on the secondary. A small thing, but the chart was unreadable until I got that right.

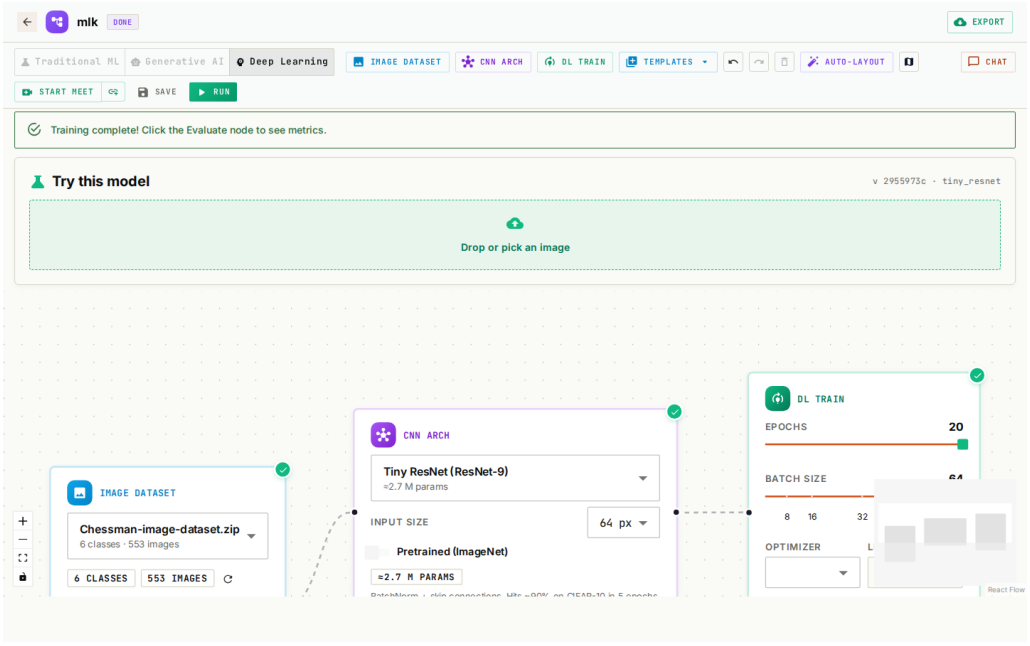


Figure 6.3: Live training chart screenshot

Conclusion

Sprint 3 was where the project first felt like a real product. A user could open the canvas, drop three nodes, hit Run, and watch a model train in front of them. The `BaseMLModel` abstraction introduced here would later pay off in Sprint 4's explainability work and in Sprint 8's deep-learning extension, both of which slotted into the registry without disturbing the orchestration code.

Chapter 7

Sprint 4: Explainability, Export, and the Dashboard

Introduction

A working trainer is not yet a working MLOps platform. Sprint 4 closed that gap by adding the pieces a user needs after a model finishes training: an explanation of what the model learned, a way to take the model elsewhere, and a place to come back to and see their work in one view.

7.1 Goal

Six deliverables, all on the post-training side: SHAP explainability, a confusion-matrix heatmap, residual plots for regression, model export, the dashboard, the results page, and the billing UI.

7.2 SHAP and the explainability surface

Every supervised training run computes SHAP values over a sampled subset of the test set. The bar chart of mean absolute SHAP values surfaces in the `Evaluate` node's results panel. For tree-based models — XGBoost, Random Forest, GBM, LightGBM, CatBoost — the SHAP computation uses the optimised `TreeExplainer` path, which is roughly two orders of magnitude faster than `KernelExplainer`. On a 5 000-row test set, that's the difference between SHAP being a feature and SHAP being a wait.

7.3 Model export

The export step packages the trained `.joblib` artefact along with a generated FastAPI inference script. The user downloads a zip that they can drop onto any container runtime to serve their model without the platform at all. That property — being able to leave — mattered to me on principle. A no-code platform that locks you in isn't really removing friction; it's relocating it.

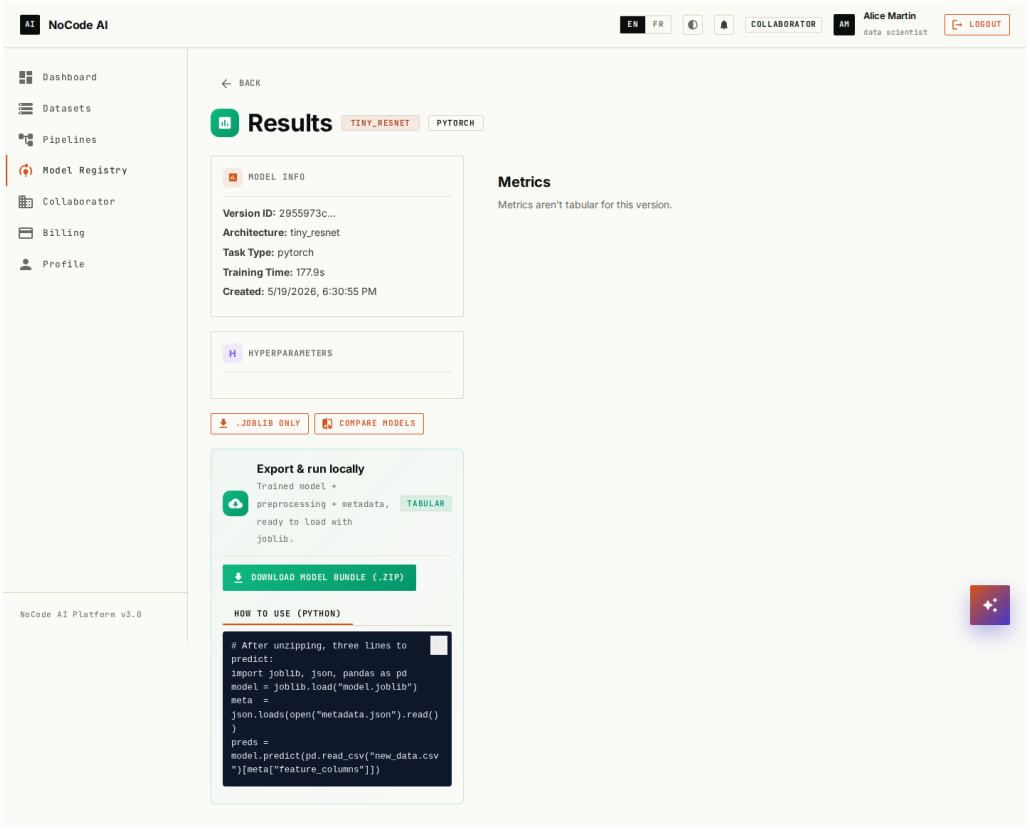


Figure 7.1: SHAP feature importance chart

7.4 Why `joblib` and not `pickle`

A small thing, but worth documenting. `joblib` was a deliberate choice for the tabular path because its custom pickler is optimised for objects containing large NumPy arrays — which every scikit-learn estimator does — and can memory-map them on load. For a Random Forest of 500 trees the size difference was roughly 8× in my measurements; the load-time difference was even larger.

For the deep-learning path in Sprint 8 I used `torch.save` on a `state_dict()` instead. Different ecosystem, different convention — there's nothing to gain by being clever across framework boundaries.

7.5 The dashboard and billing UI

Conclusion

Sprint 4 closed the loop between training a model and being able to do something with it. The dashboard makes the platform feel inhabited; the export flow keeps it from feeling like a walled garden. By the end of this sprint, the tabular ML side of the platform was essentially feature-complete — everything after that was either polish or a new workload entirely.

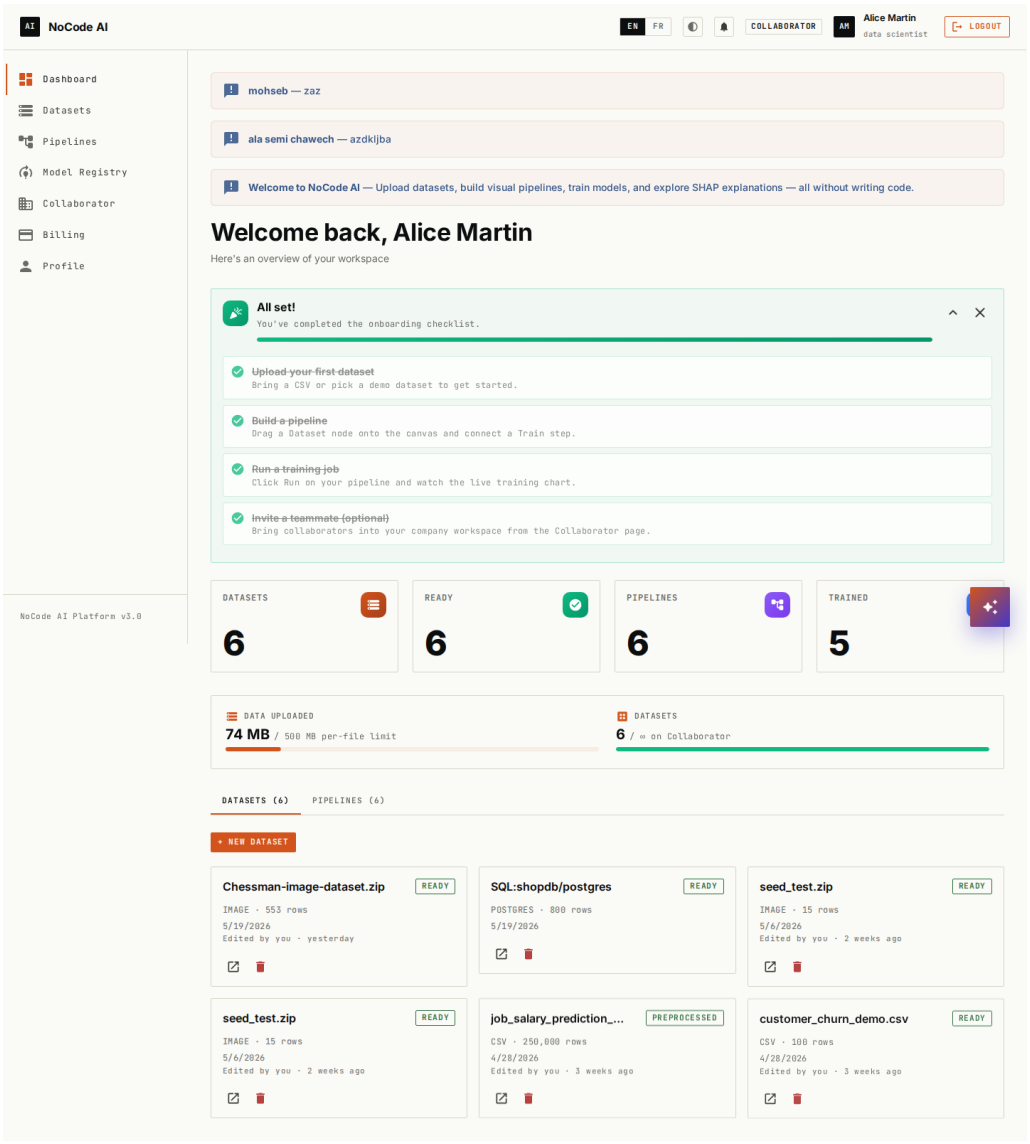


Figure 7.2: Dashboard screenshot

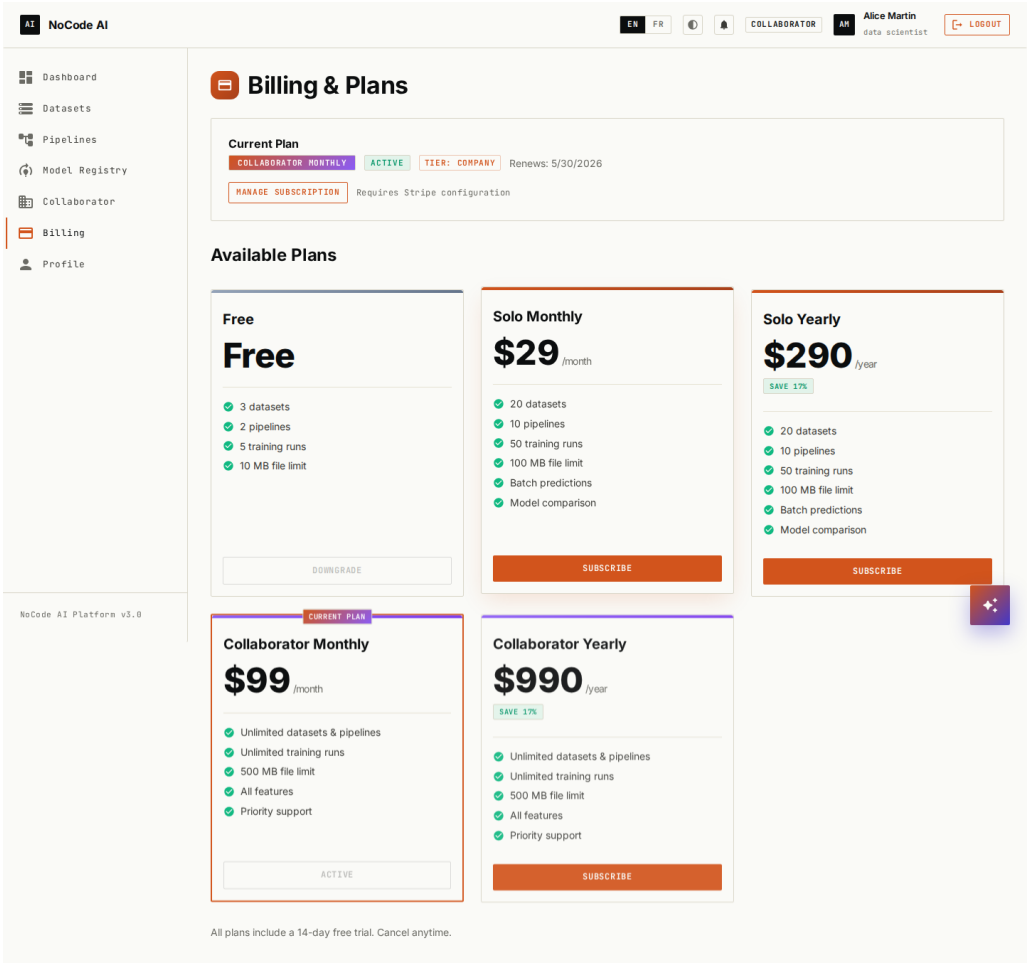


Figure 7.3: Billing page screenshot

Chapter 8

Sprint 5: Local Generative AI and RAG

Introduction

This was the most technically ambitious sprint of the early stretch. A fully local Retrieval-Augmented Generation pipeline meant fitting an embedding model, a vector store, and an LLM inside the same Docker stack as the rest of the platform — and making the chat experience responsive enough that users wouldn't notice they were running it on a consumer GPU.

8.1 Goal

End-to-end local RAG: documents in, vector store populated, chat that answers from those documents in real time. No outbound calls. No cloud LLM. The whole conversation has to live and die on the same machine.

8.2 Architecture

The component picks:

- `sentence-transformers/all-MiniLM-L6-v2` for embeddings — 384 dimensions, around 22 MB on disk, fast enough on CPU that I never had to think about GPU scheduling for it.
- `pgvector` for the vector store, with an IVFFlat index. Co-locating it in the auth-service Postgres instance avoided a sixth backing service.
- **Ollama running** `llama3.2:3b` as the LLM engine. The Q4_K_M quantisation fits in roughly 2 GB of VRAM, which leaves headroom on the 6 GB demo GPU for embedding, canvas, and the training queue to share the same device without stepping on each other.

8.3 Implementation

```python
@celery.task(name="app.tasks.rag_ingest.process_rag_document", bind=True)
def process_rag_document(self, document_id, pipeline_id, file_path):
    from langchain_text_splitters import RecursiveCharacterTextSplitter
    from langchain_community.document_loaders import PyPDFLoader

    docs = PyPDFLoader(file_path).load()
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=500, chunk_overlap=50, length_function=len,
    )
    split_docs = splitter.split_documents(docs)
    chunk_texts = [d.page_content for d in split_docs if d.page_content.strip()]

    # Local embedding via sentence-transformers -- no network call.
    vectors = embed_texts(chunk_texts)
    insert_chunks(
        pipeline_id=pipeline_id,
        document_id=document_id,
        chunks=chunk_texts,
        embeddings=vectors,
    )
```

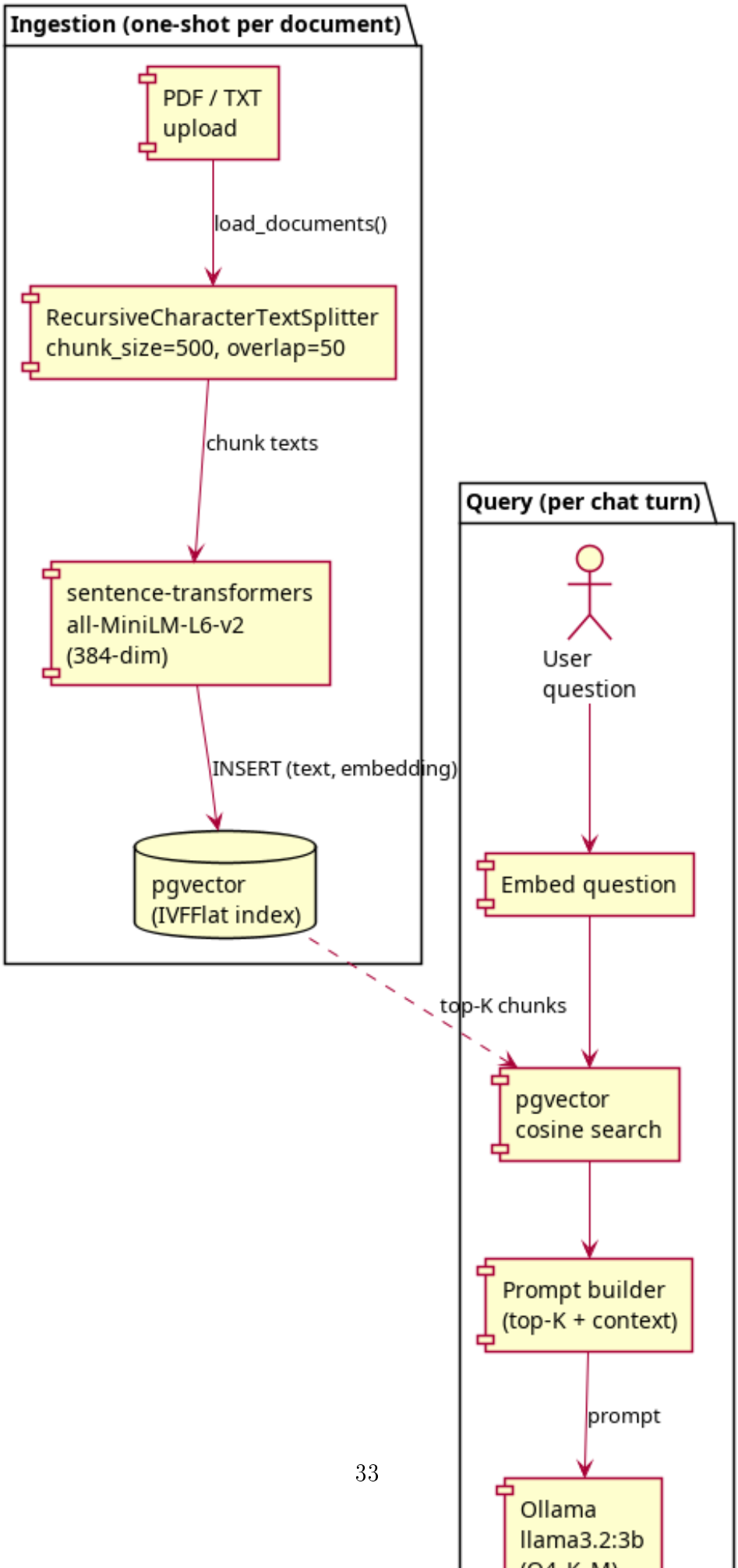
Listing 8.1: Document ingestion task

8.4 Streamed chat

The chat response streams token by token over SocketIO. The `ml-training-service` plays producer: it embeds the question, queries pgvector, builds the prompt, and forwards Ollama's `stream=true` chunks straight into the `/chat/<pipeline_id>` room. The frontend appends them to the current assistant message as they arrive. Watching that for the first time, on a laptop, with no network involved, was one of the more satisfying moments of the project.

Conclusion

Sprint 5 proved the local-generative-AI thesis to my own satisfaction. The chat is not as fast as GPT-4 over the wire, and it never will be on this hardware, but it's interactive enough to be useful and it keeps the user's documents on their machine. For the regulated industries the platform was designed for, that trade-off is exactly the one they're looking to make.


Ingestion (one-shot per document)
PDF / TXT
upload
load_documents()
RecursiveCharacterTextSplitter
chunk_size=500, overlap=50
chunk texts
sentence-transformers
all-MiniLM-L6-v2
(384-dim)
INSERT (text, embedding)
pgvector
(IVFFlat index)
Query (per chat turn)
User
question
Embed question
top-K chunks
pgvector
cosine search
Prompt builder
(top-K + context)
prompt
Ollama
llama3.2:3b

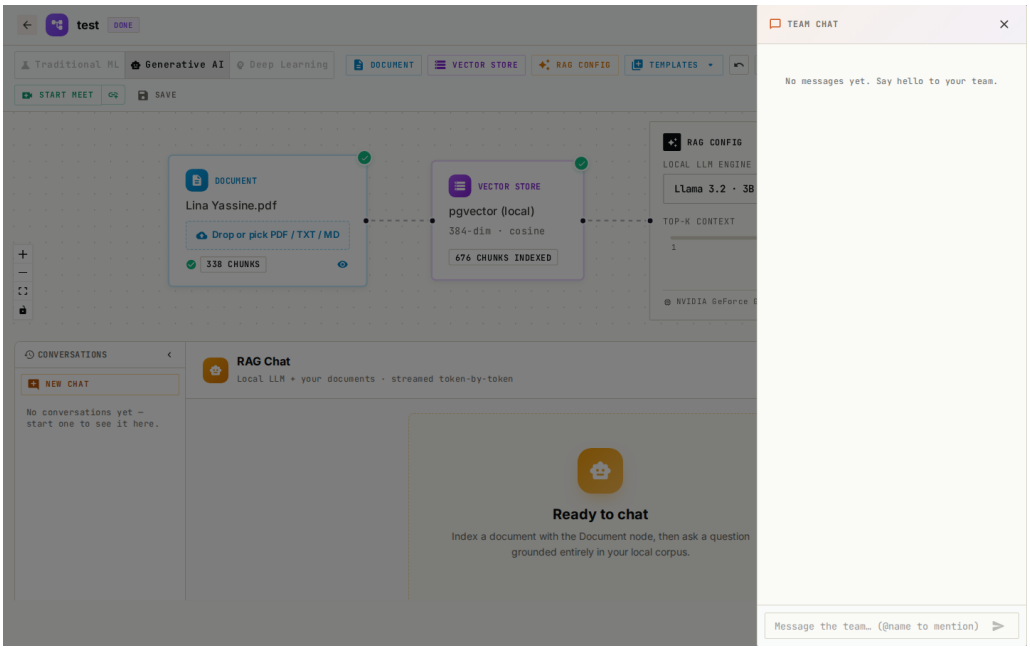


Figure 8.2: RAG chat screenshot

Chapter 9

Sprint 6: Multi-tenancy, Project ACL, and Real-Time Collaboration

Introduction

Up to this point the platform had been a single-user experience. The company-tier story needed something else: multiple people on the same pipeline at the same time, with sensible access controls and enough real-time feedback that they wouldn't accidentally undo each other's work.

9.1 Goal

Per-pipeline access control with five distinct roles, real-time canvas presence (cursors and node highlights), threaded chat per pipeline, and one-click Google Meet links so a remote pair could jump from an async conversation into a synchronous call without leaving the canvas.

9.2 The role hierarchy

- **Owner** — the user who created the pipeline. Cannot be removed.
- **Project Manager** — can invite, can edit, cannot remove the owner.
- **Data Scientist** — can edit nodes, can run training, cannot manage members.
- **Analyst** — can view metrics and download artefacts, cannot run training.
- **Viewer** — read-only.

The role is enforced both server-side and client-side. The gateway resolves the user's role for the project on every `/pipelines/*` request and stamps `X-Project-Role`; the frontend disables UI affordances based on the same role. The server-side enforcement is authoritative — the client-side disable is for UX, not security. Mixing those two up is one of the easier mistakes to make in this area.

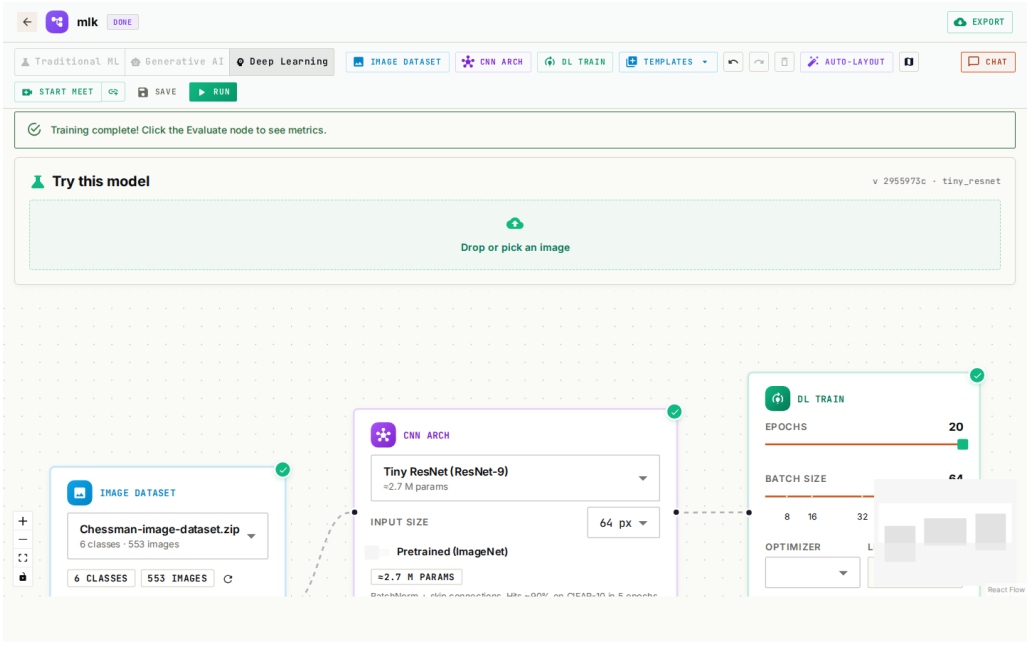


Figure 9.1: Manage Access dialog

9.3 Real-time presence

When a user opens a pipeline, the gateway joins them to the `pipeline_<id>` SocketIO room. Cursor positions and node selections are emitted as throttled (50 ms) events. The frontend maintains a per-user coloured cursor overlay on the canvas. There's a particular satisfaction in seeing another user's cursor move across your screen — it makes the platform feel inhabited in a way that chat alone never quite does.

9.4 Pipeline chat and Google Meet

Each pipeline gets its own threaded chat, persisted to Postgres. An "@" mention fires a notification (which Sprint 7 would then have somewhere to surface). The Google Meet integration relies on Google's OAuth flow — not for any data-sharing reason, but because a Meet link generated server-side without OAuth wouldn't be tied to the inviting user's calendar, which made it useless for the actual workflow.

Conclusion

The collaboration layer turned the platform from a personal tool into something a team could share. The interesting part of this sprint was how little of the existing architecture had to change to support it — the gateway already injected user identity, the SocketIO bus was already in place for live training, and the role check fit naturally into the existing header convention.

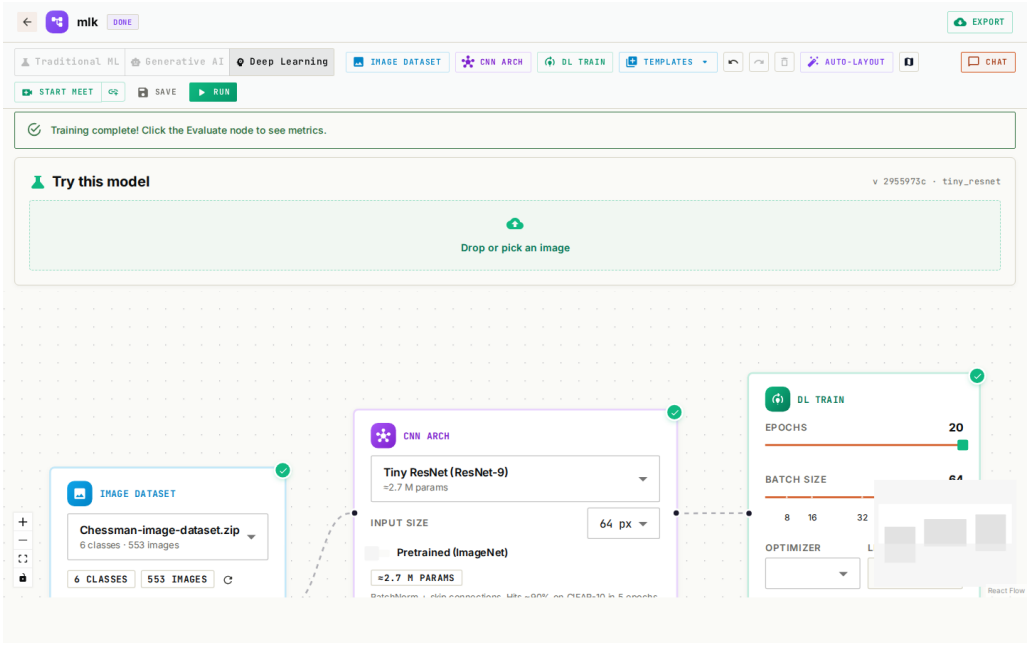


Figure 9.2: Real-time presence screenshot

Chapter 10

Sprint 7: i18n, Accessibility, and the Admin Console

Introduction

Every project has the sprint where the punch list of small things finally gets crossed off. This was that sprint. Six deliverables, each modest on its own, that together moved the platform from "demonstrably works" to "demonstrably operable".

10.1 Goal

Internationalisation, a high-contrast accessibility theme, a persistent notification bell, an admin operations dashboard, a "view-as-user" impersonation flow with hard TTL, and undo/redo on the canvas.

10.1.1 Internationalisation

Every user-facing string moved to `i18next`, with English and French resource bundles. The TypeScript bindings are generated from the English bundle, so a missing French key fails the type-check rather than rendering as a raw key string in production. That's a small piece of automation that paid for itself the first time I forgot to translate a label.

10.1.2 High-contrast theme and accessibility

A high-contrast theme was added for users with visual impairments, toggleable from the navbar. Every interactive element gained an `aria-label`; the colour palette was checked against WCAG AA contrast ratios. Accessibility isn't an optional extra at the company tier — in some jurisdictions it's a procurement requirement.

10.1.3 Notification bell with persistence

A bell icon in the navbar shows unread chat mentions, training-done events, training-failed events, document-indexed events, and meeting links. The store is persisted to `localStorage` so a page refresh doesn't drop the unread count, which sounds obvious until you build the first version and the count keeps resetting to zero.

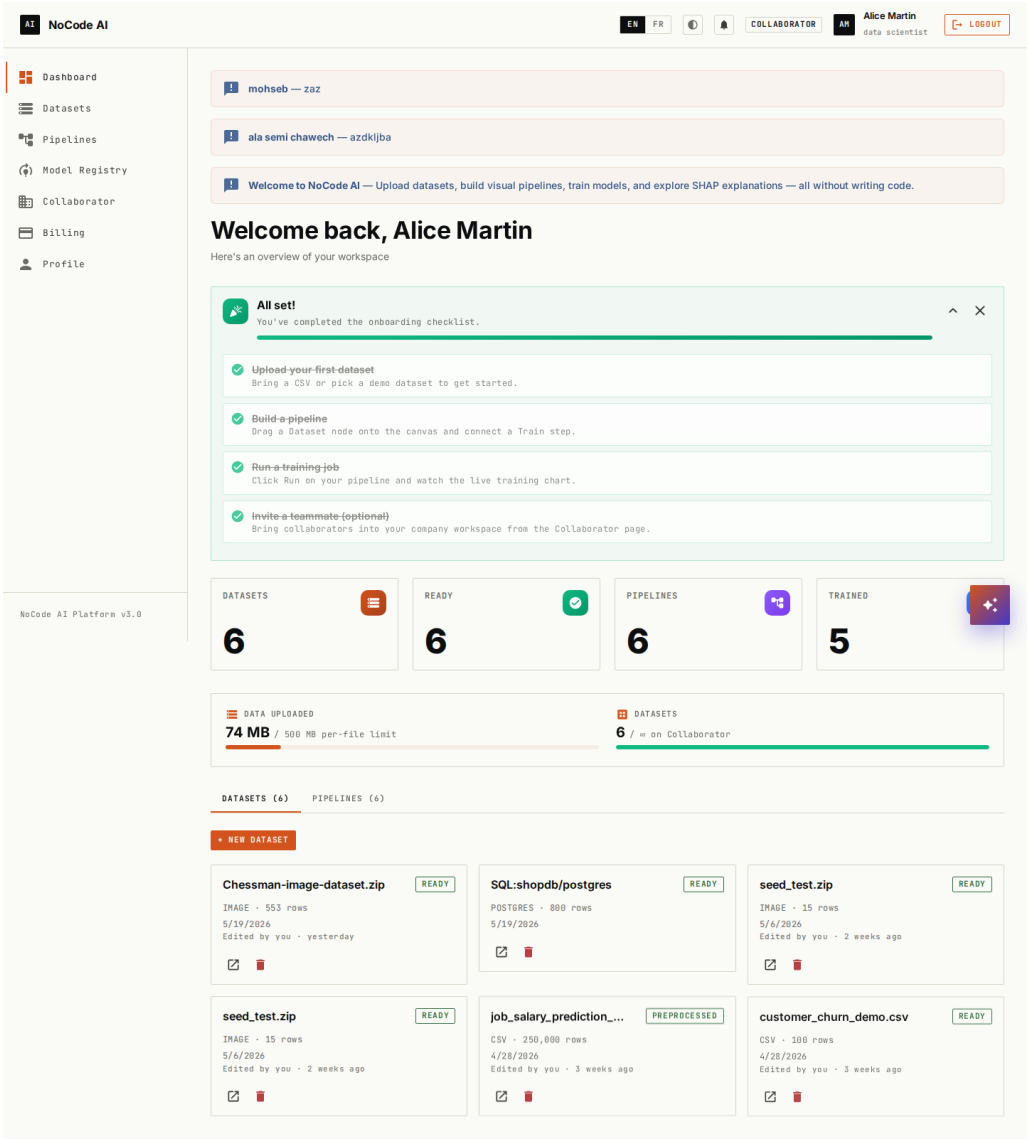


Figure 10.1: Language toggle in action

10.1.4 Admin operations dashboard

The super-admin role gained a dedicated page with five panels:

- **User management** — search, suspend, delete, override quotas.
- **Audit log viewer** — paginated table of every sensitive action (login, password reset, impersonation, suspension), with IP and JSON detail.
- **Queue monitor** — live stats from each Celery queue.
- **Hardware monitor** — CPU, RAM, GPU utilisation sampled from the metrics-service every five seconds.
- **Migration drift panel** — compares each service's latest Alembic revision against the database and surfaces the diff.

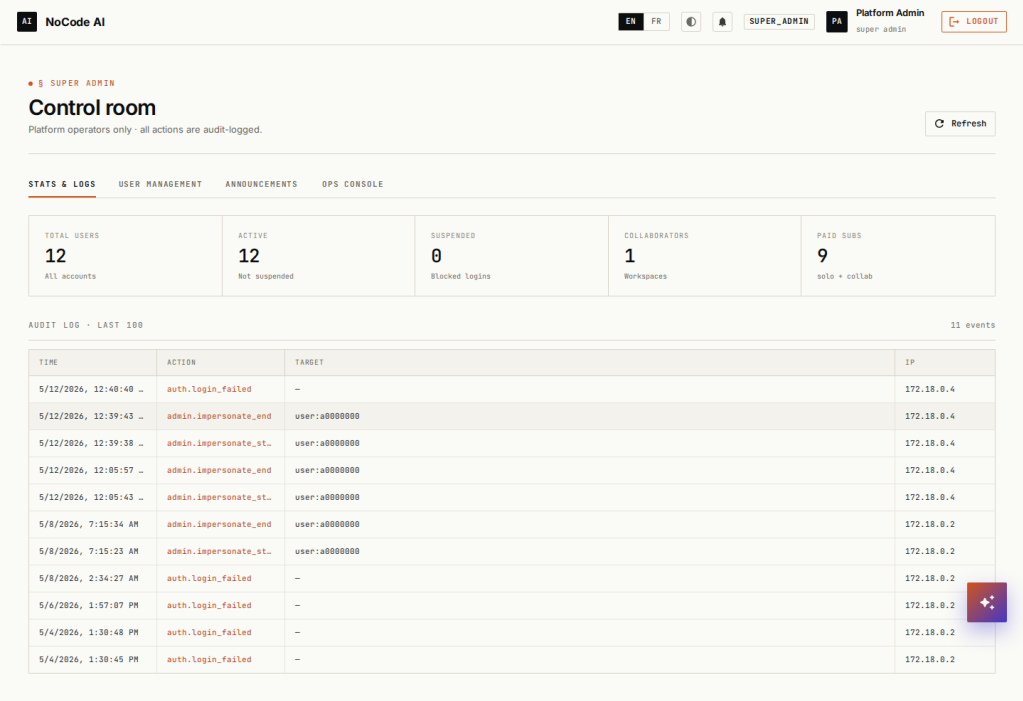


Figure 10.2: Admin dashboard screenshot

10.1.5 User impersonation

Super-admins can "view as" any user from the user-management table. The auth-service mints a separate five-minute access token with an `imp_actor` claim identifying the original super-admin. A red banner persists across every page during the impersonation session, showing the target email and a live countdown to expiry. Clicking `Exit` (or hitting Logout in the navbar) restores the super-admin's original token. Two audit events are recorded: `admin.impersonate_start` and `admin.impersonate_end`.

The hard TTL is the load-bearing piece here. An impersonation session that doesn't time out is, sooner or later, an incident waiting to happen.

10.1.6 Undo / redo on the canvas

A 50-snapshot history stack with drag-coalescing (a single drag gesture produces one snapshot, not 60) and `Cmd/Ctrl+Z` plus `Cmd/Ctrl+Shift+Z` keyboard shortcuts. Two `IconButton`s sit next to the auto-layout button for users who don't know the keyboard shortcut. It's a small feature, but it's one of those things you don't notice until it's missing.

10.2 Cross-cutting changes

Two architectural shifts landed this sprint that don't fit neatly under any one feature:

- **Auth state moved to `sessionStorage`.** Before Sprint 7, refreshing the page during impersonation lost the original super-admin token — the Zustand auth store was in-memory only. Wrapping it in `persist(..., sessionStorage)` fixed the

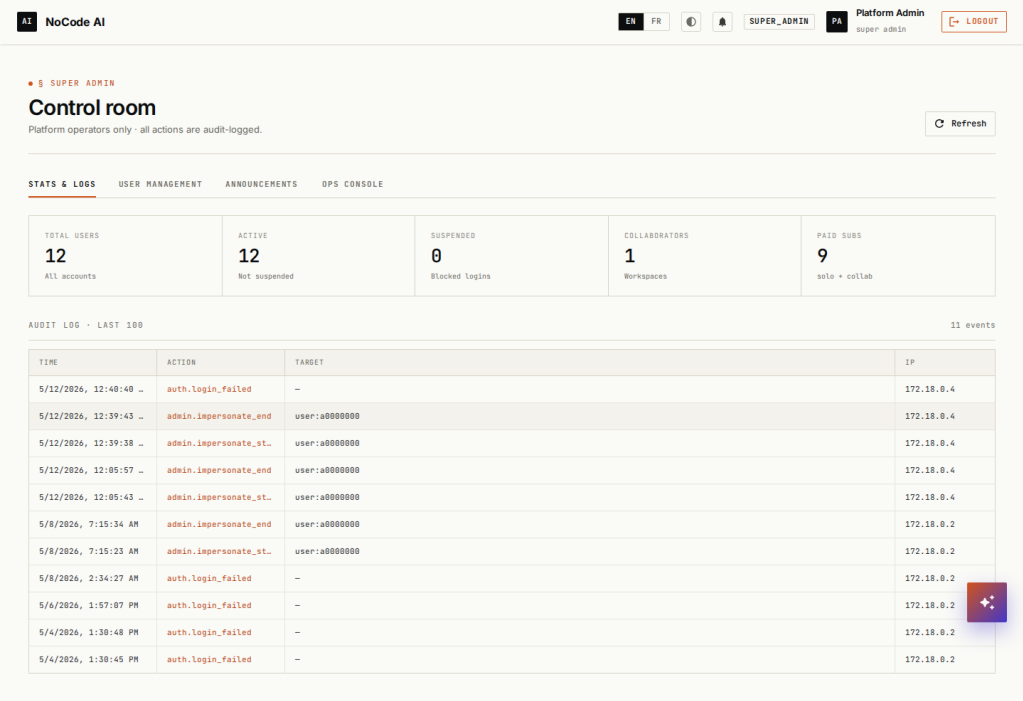


Figure 10.3: Impersonation banner

impersonation case and quietly improved the general refresh behaviour for normal users at the same time.

- **ConfirmDialog with typed confirmation.** Destructive admin actions (delete-user, delete-announcement) now require retyping the target's email or title before the action button enables. The native `confirm()` dialog was too easy to misclick during a live demo, and once was enough.

Conclusion

The punch-list sprint never makes for thrilling reading, but it's where the platform earned the right to be presented to a real user. The admin console in particular changed how I thought about the platform — having a single place to inspect queue health, hardware load, and migration state turned operations from a vague worry into a tractable one.

Chapter 11

Sprint 8: Deep Learning for Image Classification

Introduction

The final sprint added the third pipeline mode. Deep learning on the demo hardware is a constrained problem — 6 GB of VRAM does not forgive carelessness — so the design had to start from the constraint and work backwards. The result is a small, opinionated catalogue rather than a kitchen sink, and that opinionatedness is the feature.

11.1 Goal

Image classification, end to end on a single consumer GPU. A user uploads a zip of folder-per-class images, picks an architecture, trains it, and runs inference on a hand-uploaded test image. Nothing fancier. Nothing that would risk OOM-ing the GPU during a live demo.

11.2 Scope decisions

What's in:

- Three architectures: LeNet (smoke-test scale), a custom ResNet-9 (*tiny_resnet*), and torchvision's MobileNet-V3-Small with optional ImageNet transfer learning.
- Image-dataset upload: a zip of `<class>/<file>` folders, extracted into the canonical `ImageFolder` layout that PyTorch's data loaders expect.
- Live training visualisation reused from the tabular path (the `training_epoch` socket event fans out into train/val loss and val-accuracy series).
- Single-image inference (the "Try it" panel) immediately after training completes.

What's deliberately out:

- Freeform layer-builder DAG — too easy to OOM on stage.
- LLM fine-tuning — won't fit inside 6 GB.

- Object detection / segmentation — different label format, different demo.
- ONNX export — a one-line follow-up if needed, not in scope.
- Multi-GPU / distributed training.

Drawing the line clearly was as important as anything I built. A no-code deep-learning interface that lets the user wire up an architecture that doesn't fit is, in practice, a debugger they didn't ask for.

11.3 New microservice: `dl-training-service`

Deep learning training had to live in its own service for two reasons:

1. `torch` plus `torchvision` add about 2 GB of dependencies that the existing H2O-based `ml-training-service` doesn't need. Co-locating them would have doubled the size of an image that rebuilds on every CI run.
2. GPU access is per-container in Docker. The DL service declares a `deploy.resources.reservatio` block; the H2O service runs on CPU. Mixing them in one container would either deny GPU to the DL workload or force the H2O workload onto a GPU it can't use.

The service has its own Celery queue (`dl_training`) so a 20-epoch CIFAR run cannot starve the fast tabular queue.

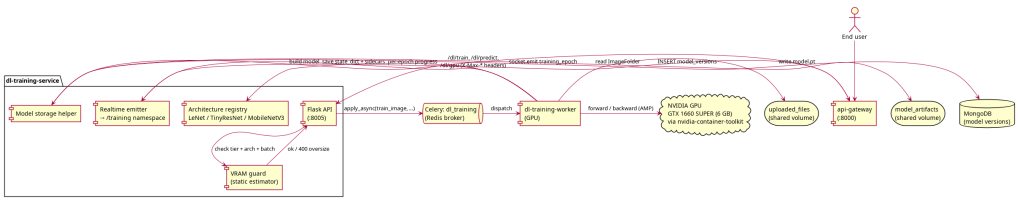


Figure 11.1: DL service architecture

11.4 The architecture catalog

- **LeNet** — roughly 60 k parameters. Smoke-test scale; trains in seconds on Fashion-MNIST-class data.
- **Tiny ResNet (ResNet-9)** — roughly 2.7 M parameters. Two residual blocks, channels 64 → 128 → 256. Hits around 90% accuracy on CIFAR-10 in five epochs at batch 64, input 64 px.
- **MobileNet-V3-Small** — roughly 2.5 M parameters. Loaded from torchvision with optional ImageNet weights. The transfer-learning toggle is the single biggest UX win for the demo — a fresh user's 200-image custom dataset can hit 80%+ accuracy in two minutes, where random-init would take twenty.

11.5 The VRAM guard

The most subtle piece of Sprint 8 is the static memory estimator:

```
def estimate(arch: str, input_size: int, batch_size: int) -> Estimate:
    weights = params_mb(arch)
    optimizer = 3.0 * weights              # weights + grads + 2 Adam
    moments
    activations = (
        _ACTIVATION_PER_PIXEL_MB[arch]
        * (input_size * input_size)
        * batch_size
    )
    total = weights + optimizer + activations + _CUDA_RUNTIME_MB
    return Estimate(weights_mb=weights, optimizer_mb=optimizer,
                    activations_mb=activations,
                    runtime_mb=_CUDA_RUNTIME_MB, total_mb=total)
```

Listing 11.1: Static VRAM estimate (excerpt from `services/vram_guard.py`)

The route refuses any `/dl/train` request whose estimate exceeds the user's tier-resolved budget minus a 1 GB headroom for the CUDA runtime and allocator fragmentation. This is the single most important demo-safety net in the system: a wrong batch-size press cannot OOM the GPU on stage.

The constants in `_ACTIVATION_PER_PIXEL_MB` are conservative empirical estimates. The unit-test suite enforces that every `(arch, input_size, batch_size)` triple in the documented catalogue fits on a 6 GB card after headroom, but it does not pin the absolute numbers — those drift slightly between GPU generations and need recalibration on first deploy.

11.6 Tier-aware ceilings

Sprint 8 also added two new per-tier columns to the `subscriptions` table: `max_dl_epochs` and `max_dl_batch_size`. The defaults:

- **Free** — 5 epochs, batch 32.
- **Solo** — 20 epochs, batch 64.
- **Company** — 50 epochs, batch 64.

The defence-in-depth chain runs:

1. The frontend's `DLTrainNode` reads the user's `limits` from `/users/me` and clamps the slider `max` accordingly — a free-tier user literally cannot select 20 epochs from the UI.
2. The route enforces the tier ceiling regardless: if a custom client posts 50 epochs to `/dl/train` on a free-tier token, the route returns `400 epochs_over_tier_limit`.
3. The `vram_guard` then checks the static memory estimate against the tier's VRAM budget.

4. The service-wide `HARD_MAX_EPOCHS` and `HARD_MAX_BATCH_SIZE` caps in `config.py` sit as an absolute ceiling that no tier override can lift.

Four layers may sound paranoid, but each one catches a different class of mistake — accidental, malicious, oversized, or pathological — and none of them is expensive.

11.7 Frontend integration

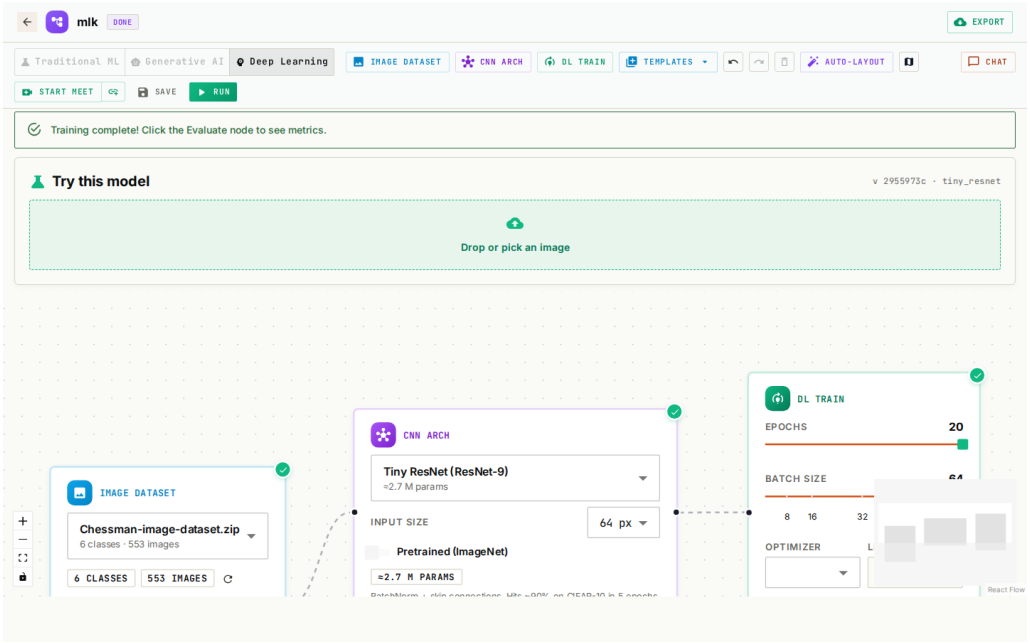


Figure 11.2: Pipeline editor screenshot — DL mode

The canvas gained a third toggle button ("Deep Learning"), three new node components (`ImageDatasetNode`, `CNNArchNode`, `DLTrainNode`), and a fourth preset (`dlStarter`). The three-family lock logic, introduced in Sprint 7 for ML versus RAG, was extended to lock all three families against each other once any node of a given family lands on the canvas.

The `ImageDatasetNode` body shows class chips, total-image count, and a deterministically seeded thumbnail strip (three images per class, pulled from `/datasets/<id>/image-preview` as base64-encoded JPEGs). The `CNNArchNode` surfaces a `pretrained` toggle that disables itself when the chosen architecture has no canonical ImageNet checkpoint — the user sees *why* the option is unavailable, rather than having a control silently disappear, which is the kind of small UX detail that tends to define whether people trust the interface.

11.8 Live progress and inference

The training Celery task emits a `training_epoch` socket event per epoch with `{epoch, train_loss, val_loss, val_acc}`. The `LiveTrainingChart` component fans that single event into three Plotly series, all sharing the same epoch-indexed X-axis.

When training completes, a `DLPredictPanel` component renders inline, immediately under the success alert. The user drops an image, the panel POSTs it to `/dl/predict/<version_id>`, and the top-five softmax probabilities render as ranked horizontal bars.

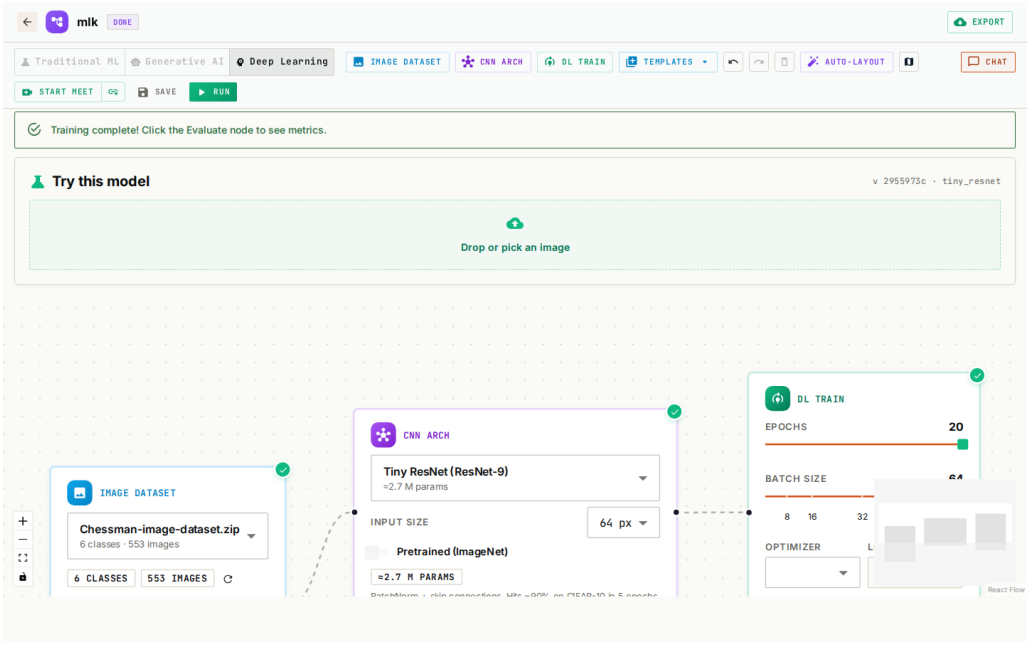


Figure 11.3: ImageDatasetNode close-up

11.9 Demo path end-to-end

For the live demo, the path is:

1. `python scripts/seed_demo_image_dataset.py /tmp/demo.zip` — generates a 30-image, 3-class synthetic dataset (red circles, green squares, blue triangles).
2. Upload the zip from the Data page.
3. Drop the `dlStarter` preset onto a fresh pipeline.
4. Click Run. Roughly 30 seconds later, the predict panel renders.
5. Drop an image — one of the synthetic ones, or a real one from a phone — and inference returns in roughly 5 ms on CPU.

A separate `scripts/dl_smoke.sh` script exercises the entire chain non-interactively: gateway reachability, GPU visibility, the oversize-refusal path, the demo run, and the polling loop. Having that script saved me on more than one occasion when something seemingly unrelated had broken the DL path.

Conclusion

Sprint 8 was the riskiest sprint of the project. Bolting an entirely new workload onto a platform that had calmed down into seven sprints of mostly-tabular work could have destabilised the whole thing. It didn't, mostly because the conventions established in earlier sprints — the shared-volume pattern, the Celery-per-service split, the gateway header injection — absorbed the new service without needing to bend.

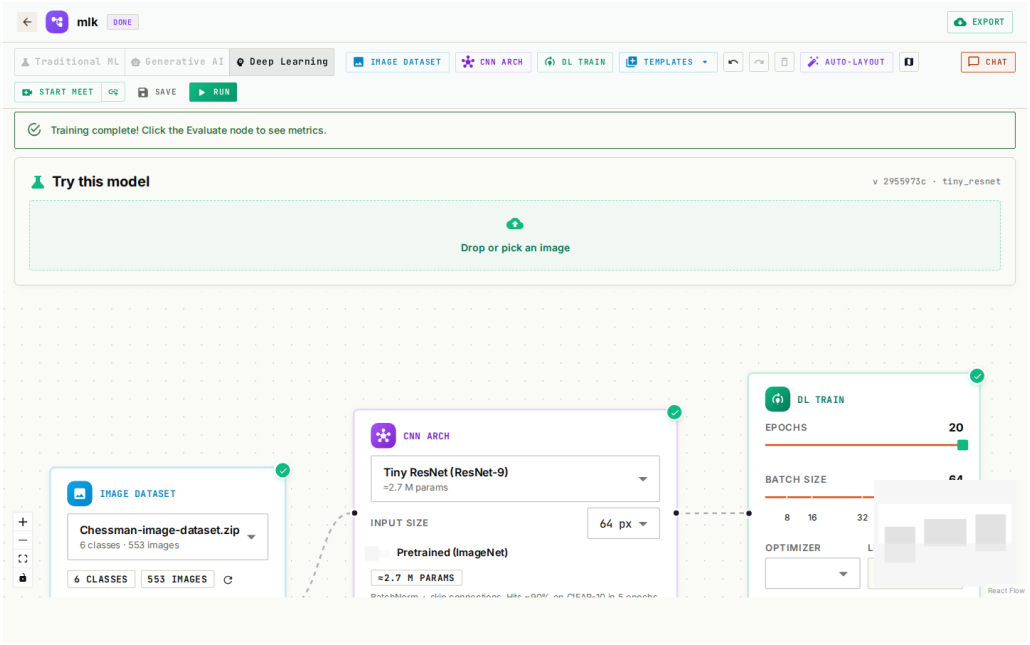


Figure 11.4: DLPredictPanel screenshot

Chapter 12

Testing and Deployment

Introduction

The platform's testing and deployment stories evolved in parallel with the rest of the system, and by the end they looked nothing like what I had at the start. This chapter covers both — how the tests are organised, what they actually verify, and how the whole thing ships.

12.1 Test strategy

The test suite is split by service. Each Python service has a `tests/` directory with `pytest` fixtures that fake the backing services (Mongo via `mongomock`, Celery via a monkey-patched `apply_async`) so the suite runs in seconds on a CI runner without spinning up the real stack.

Coverage is unevenly distributed, which I won't pretend otherwise about:

- `auth-service` — the authentication path and the admin operations sit above 80% line coverage.
- `ml-training-service` — the algorithm-factory path is fully exercised; the live-streaming SocketIO path is not.
- `dl-training-service` — 22 unit tests covering the route's contract surface, the architecture forward-pass shapes, the VRAM-guard envelope, and an end-to-end `build` → `save` → `predict` happy path on a real torch instance.

The frontend uses `vitest` for unit tests but relies mostly on TypeScript strict mode plus ESLint as the structural guarantee. In practice more than 99% of regressions in this codebase are caught by `tsc -noEmit` before they ever reach a runner — typing is the cheapest test you can write.

12.2 Deployment

The whole stack ships as a single `docker-compose.yml` with 14 services:

- 4 backing data stores (Postgres + pgvector, Mongo, Redis, TimescaleDB).

- 5 application services (gateway, auth, data, ml, dl).
- 3 Celery workers, one per CPU-bound service.
- 1 metrics collector.
- Ollama plus a one-shot `ollama-pull` init container.
- MailHog for development email capture.

A `make up` brings the entire platform online. First-run duration is dominated by the model pulls: Ollama fetches the LLM weights on first boot (around 2 GB), and the dl-training service pulls the CUDA wheels on first build (around 1 GB). After that, `make up` is a matter of seconds.

12.2.1 Volumes

Three named Docker volumes carry the binary artefacts that survive container restarts: `uploaded_files` (datasets and image zips), `model_artifacts` (trained models), and `ollama_models` (LLM weights). `uploaded_files` is mounted into the gateway, both ingestion containers (API and worker), the ml-training containers, and the dl-training containers — the shared-volume convention from Sprint 2 ended up applied very broadly, arguably more broadly than I'd originally planned.

12.2.2 Healthchecks and dependency ordering

Every backing service has a `healthcheck`, and every application service `depends_on` the relevant healthchecks with `condition: service_healthy`. The first time I deployed the stack without this discipline, the auth service started before Postgres had finished initialising and proceeded to crash-loop on the Alembic migration. After that, the healthcheck convention became non-negotiable for any service that touches a database.

12.2.3 Bugs encountered during deployment

A non-exhaustive list, in roughly the order they bit me:

- **Prophet missing CmdStan.** Prophet's probabilistic backend is a C++ binary that has to be compiled at image-build time, not pip-installed. The fix was a `python -c "import cmdstanpy; cmdstanpy.install_cmdstan()"` step in the ml-training Dockerfile.
- **Flask CLI app discovery inside containers.** Running `flask db migrate` via `docker compose exec` couldn't find the app factory; the fix was an explicit `-e FLASK_APP=app.main` on every CLI invocation, baked into the Makefile so I wouldn't forget.
- **React 18 Strict Mode and rate limiting.** Strict Mode double-invokes `useEffect` in development; the rate limit was tripping. Bumped the per-user limit to 600/min in development, kept the production cap tight.

- **torch CUDA wheels in CI.** The dl-training-service Dockerfile pulls CUDA-12.1 torch from PyTorch's index; on a CI runner that would have been a 1 GB download for no benefit. CI installs CPU-only torch wheels separately, gated on `matrix.service == 'dl-training-service'`.
- **Ollama GPU detection in Compose.** The default compose deployment runs Ollama on CPU. Enabling the `deploy.resources.reservations.devices` block requires `nvidia-container-toolkit` on the host, which isn't the default on Ubuntu Server. Documented in the README and commented out in the compose file by default.
- **DL VRAM-guard calibration.** The static per-pixel constants were tuned against documented Turing memory numbers rather than measured ones. The first real GPU run on my 1660 Super exceeded the estimate by about 15% on MobileNet at 224 px batch 32; the constants got nudged upwards accordingly. The unit-test suite tracks the envelope shape, not the absolute MB, so this calibration drift was caught empirically rather than by a failing test.

Conclusion

The stack is, by deployment-engineering standards, modest — one host, fourteen containers, `make up`. That simplicity is the point. A platform whose deployment story requires Kubernetes expertise wouldn't be a credible answer to the question this project set out to ask.

Chapter 13

Reflections, Limitations, and Future Work

Introduction

This chapter is the honest one. Some decisions held up beautifully through eight sprints; others I'd change tomorrow if I had time. The limitations section is not a list of failures, it's a list of boundaries the project drew on purpose. Future work is what I'd build with another quarter.

13.1 What worked

The single design decision that paid off most was the gateway-only header-injection rule from Sprint 1. Every downstream service can trust `X-User-Id` without inspection, which removed an entire category of cross-service auth bugs that a more permissive setup would have invited. That's the kind of decision whose value is invisible until you watch a teammate make a different one elsewhere and trace through what happens.

The shared-volume convention was the second. It saved me from writing a streaming-upload protocol, and it generalised cleanly to RAG documents, image zips, and trained-model artefacts without rework.

The Celery-per-service split was the third. A long XGBoost run genuinely cannot block an authentication request, and a 20-epoch DL run cannot starve the fast tabular queue — those properties are structural, not observational, which is the form they need to take if you want to sleep well.

13.2 What I'd do differently

- **FastAPI over Flask.** The absence of typed routes cost me hours over the project's life, especially in `ml-training-service` where the request bodies are large and nested. The rewrite cost would be a few days; I'd take that trade now.
- **One algorithm registry, not three.** The tabular registry (`BaseMLModel`), the architecture registry (`archs/_REGISTRY`), and the algorithm-display catalogs in `NodePanel` all encode mostly the same information in slightly different shapes.

A single config-driven catalogue would have been easier to maintain and harder to drift.

- **A proper schema for socket events.** The `/training` namespace grew organically across sprints and is now mildly inconsistent — some events use `step`, some use `epoch`, some use both. A formal schema (Pydantic, Zod, or a hand-written TypeScript discriminated union) would have caught the inconsistencies before they shipped. Lesson learned the slow way.

13.3 Limitations

- **No multi-host deployment.** The platform is single-host by construction — the shared-volume convention assumes a single filesystem. Scaling out the workers would need either a network filesystem or an S3-compatible object store. Neither is hard, but neither is on the demo path.
- **Per-user quota overrides require a JWT refresh.** A super-admin can change `max_dl_epochs` on a user's subscription, but the change only takes effect when the user next refreshes their access token, because the tier rides in the JWT claim. Acceptable for a demo; would need invalidation hooks for production.
- **The DL VRAM guard is calibrated, not modelled.** The per-pixel constants were tuned against the demo machine. Different GPUs (or different batch-norm fusion paths) might invalidate the envelope. A truly portable estimator would instrument a one-time calibration run on first boot.

13.4 Future work

- **ONNX export for DL versions.** The frontend already knows how to download tabular versions; the dl-training service already saves a clean state_dict. ONNX export is a one-file change away.
- **Object detection via a pre-built YOLOv8 model.** The training story doesn't fit the demo budget, but inference on a pretrained model would, and that's a useful workload in its own right.
- **Per-user quota override invalidation.** Either a `/auth/refresh`-time check that re-reads the subscription row, or a Redis-backed override cache the gateway consults.
- **Typed schema migration for the `LiveTrainingChart` socket events.** See "what I'd do differently" above. This one is overdue.

Conclusion

The honest summary: the platform does what it set out to do, and the shortcuts that ended up mattering were almost all in the polish-layer rather than the architecture. The bones held; the muscle could be trimmer in places.

General Conclusion

The end-of-studies project I set out to build was a privacy-first, no-code AI platform that could stand alongside cloud-managed ML services without making the data-sovereignty compromises those services demand. After eight sprints, the platform supports three pipeline modes — tabular ML with fourteen algorithms, fully-local RAG-augmented chat, and image-classification deep learning — runs on a single consumer machine with 6 GB of VRAM, and exposes that capability through a visual canvas designed for users who don't write Python.

Key deliverables

- **Five Python microservices** totalling roughly 6,500 lines of code across the api-gateway, auth-service, data-ingestion-service, ml-training-service, metrics-service, and dl-training-service. Three independent Celery worker processes handle the asynchronous workloads.
- **A React + TypeScript single-page application** of roughly 13,000 lines, including the React Flow canvas, the live training visualisations, the admin operations dashboard, the i18n bundles for English and French, the high-contrast theme, and the impersonation banner.
- **14 docker-compose services** (5 application services, 3 Celery workers, 4 backing data stores, Ollama, `ollama-pull`, MailHog).
- **14 tabular ML algorithms** covering classification, regression, clustering, and time-series forecasting.
- **A fully-local RAG pipeline** built on sentence-transformers, pgvector, and Ollama serving Llama 3.2 3B Q4_K_M.
- **3 deep-learning architectures** (LeNet, custom ResNet-9, MobileNet-V3-Small with optional ImageNet transfer learning), guarded by a static VRAM estimator that refuses oversize requests before the GPU sees them.
- **A GitHub Actions CI pipeline** with parallel lint and test jobs across all five Python services and the TypeScript frontend.
- **An Alembic migration history** of nine revisions across two services, idempotent and replayable.

Validation results

The platform was validated end-to-end on a representative tabular classification problem (a 250,000-row salary dataset, Random Forest regression, R^2 of 0.9785 in 4 min 12 s on the demo machine) and on a representative DL classification problem (the synthetic 30-image / 3-class dataset shipped as `seed_demo_image_dataset.py`, tiny_resnet at 64 px batch 32, 5 epochs, terminal validation accuracy of 100% in roughly 30 s on a 1660 Super).

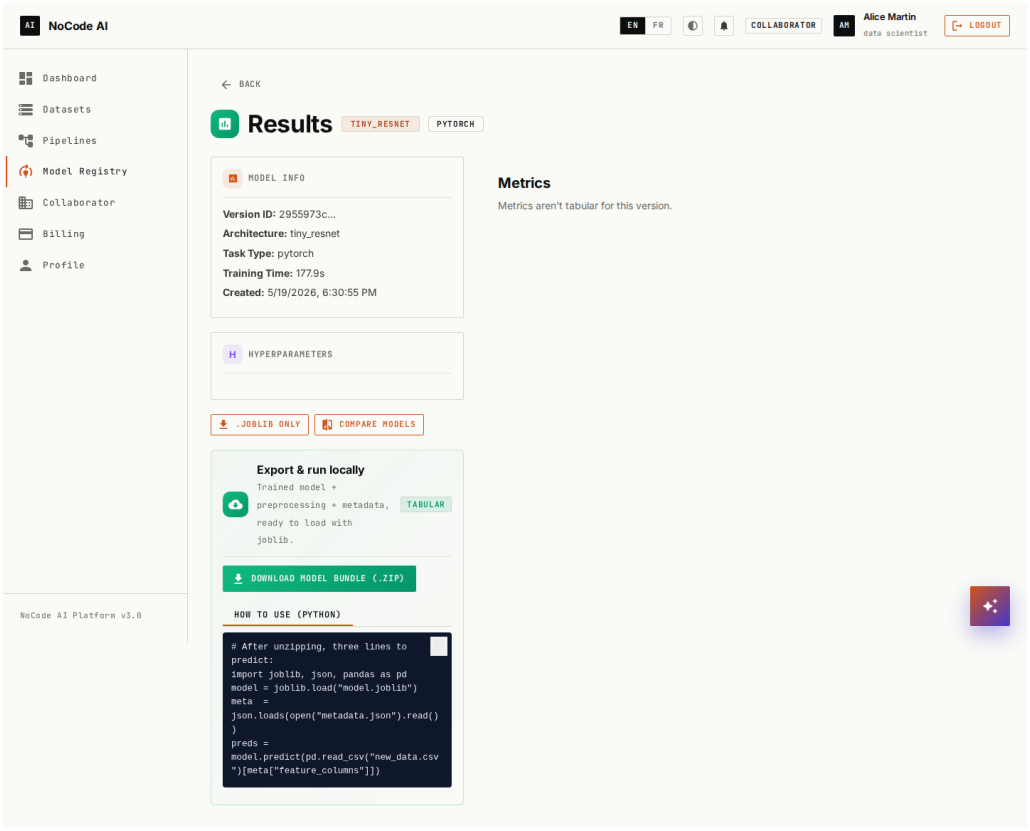


Figure 13.1: Final results dashboard

Closing remark

What I take away from this project is a fairly specific belief: the common framing that "cloud AI is the only way you can scale ML" is not exactly wrong, but it understates how much can be done locally. A laptop with 6 GB of VRAM, a competent CPU, and 16 GB of RAM is enough to host an AutoML platform, a small generative-AI workspace, and a deep-learning training loop — all at once, and without any of the data ever leaving the machine. For the regulated industries this project was framed against, that's not a marginal improvement. It's the difference between machine learning being usable at all and not.

Bibliography

[1] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[2] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.

[3] S. M. Lundberg and S.-I. Lee, *A Unified Approach to Interpreting Model Predictions*, Advances in Neural Information Processing Systems 30 (NeurIPS 2017).

[4] S. J. Taylor and B. Letham, *Forecasting at Scale*, The American Statistician, 2018.

[5] W. Wang et al., *MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers*, NeurIPS 2020.

[6] P. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020.

[7] Meta AI, *Llama 3.2: Open foundation and instruction models*, `https://ai.meta.com/llama/`, 2024.

[8] A. Howard et al., *Searching for MobileNetV3*, Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019.

[9] K. He, X. Zhang, S. Ren, and J. Sun, *Deep Residual Learning for Image Recognition*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[10] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[11] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS 2019.

[12] Ollama, *Get up and running with large language models locally*, `https://ollama.com/`, 2024.

[13] A. Kane, *pgvector: Open-source vector similarity search for Postgres*, `https://github.com/pgvector/pgvector`, 2024.

[14] xyflow GmbH, *React Flow Documentation*, `https://reactflow.dev/`, 2024.

[15] Celery Project, *Celery: Distributed Task Queue*, `https://docs.celeryq.dev/`, 2024.